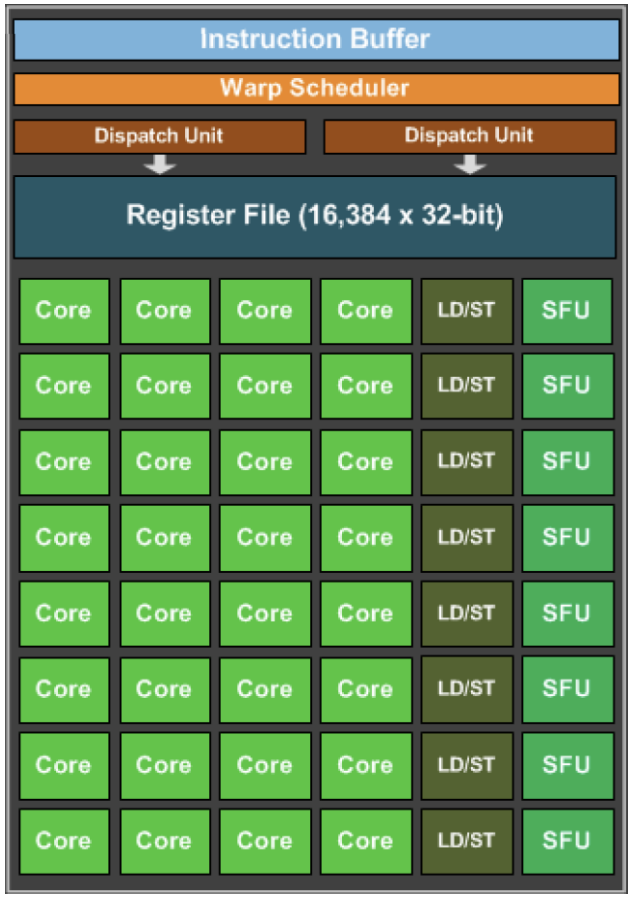
CSE113: Parallel Programming

- **Topics**:
  - GPUs



Announcements

- Last day to turn in HW 3 was this weekend, hope you get it in!
- HW 4 to be released. We will discuss it in class a little bit today.

Announcements

- Today GPUs. This is just the tip of the iceberg!

- Rest of quarter:
  - Memory models
  - Barriers
  - Concurrent Set

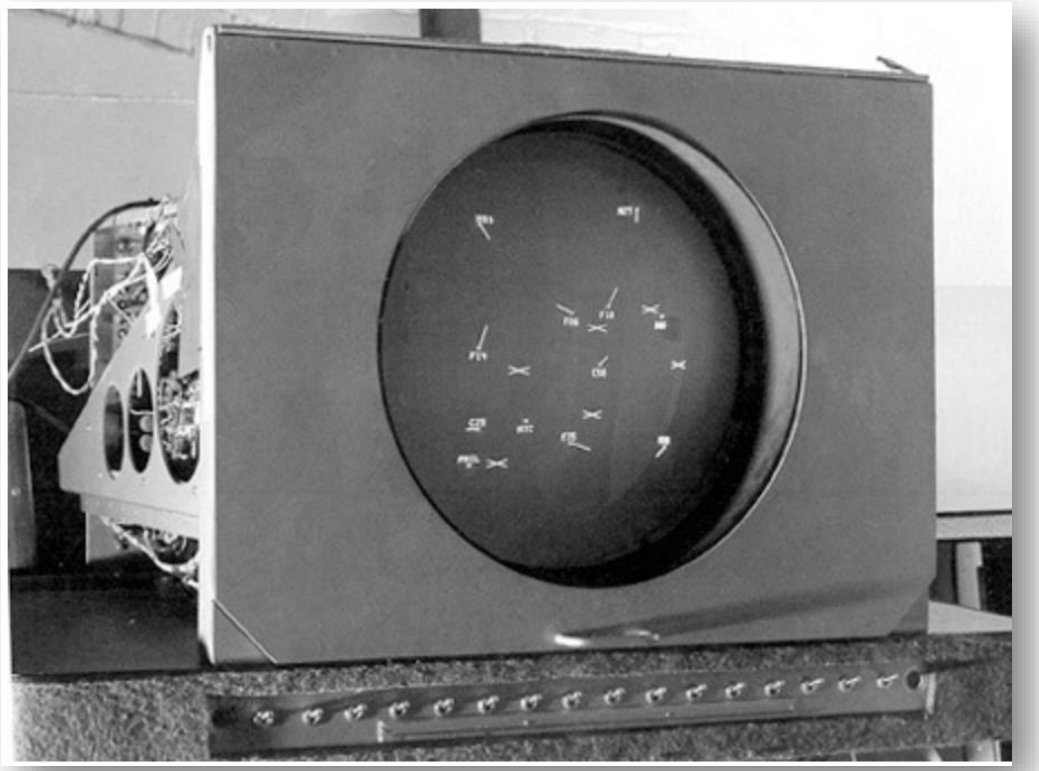
GPUs: a brief history

First chapter of CUDA by Example
https://www.techspot.com/article/650-history-of-the-gpu/

The very beginning

- **Specialized hardware** to accelerate graphics rendering

- One of the first real-time computers: **Whirlwind 1 at MIT** (1951)
  - **Flight simulator** for bombers
  - **Vector graphics**



Vector graphics are a form of computer graphics in which visual images are created directly from geometric shapes defined on a Cartesian plane, such as points, lines, curves and polygons.

Image from: https://ohiostate.pressbooks.pub/graphicshistory/chapter/2-1-whirlwind-and-sage/

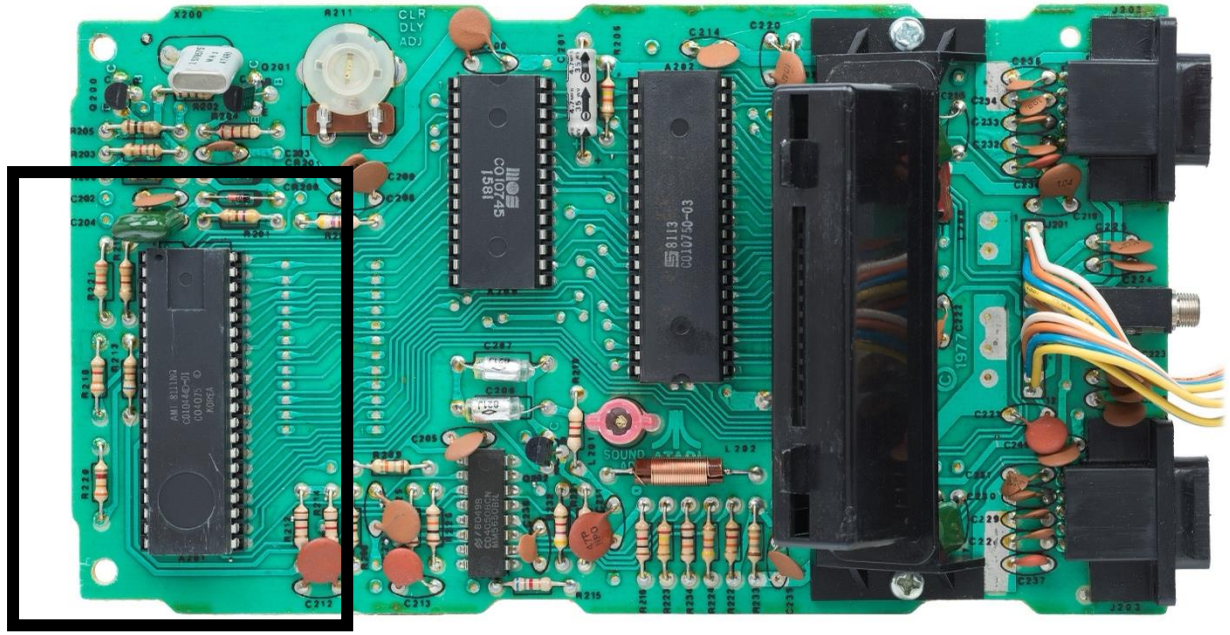
Specialization

- Next 30 years, specialized hardware for specialized software to display 2D graphics

- Specialized
  - Typically ran **specific programs**
  - Portability (i.e. multiple program) was not a top priority
  - Even the idea of portable ISAs for CPUs were not mainstream

Multi-program devices

- 1977: Television Interface Adapter
  - One of the first (and widely produced) portable GPUs





from: https://en.wikipedia.org/wiki/Television_Interface_Adaptor

River Raid Game



https://en.wikipedia.org/wiki/DirectX

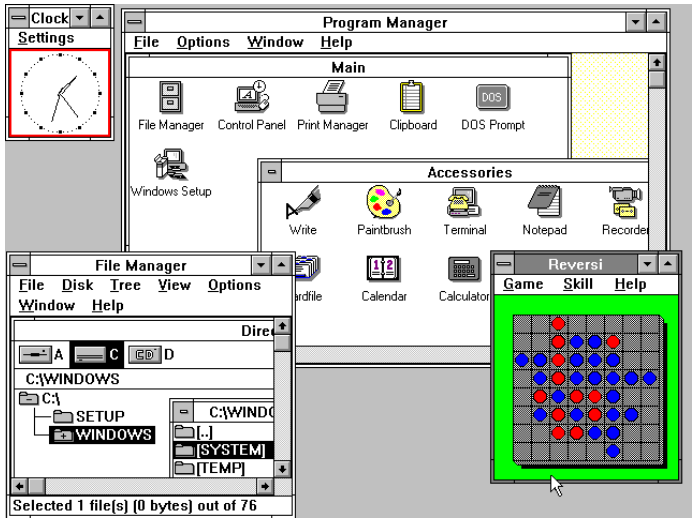
https://en.wikipedia.org/wiki/Microsoft_Windows

https://en.wikipedia.org/wiki/OpenGL

OS integration

- 1990s: Windows: a graphical operating systems, required chips to support 2D graphics.

- **New APIs** started appearing, to enable GUI programs



Windows 3 (1990)

1992

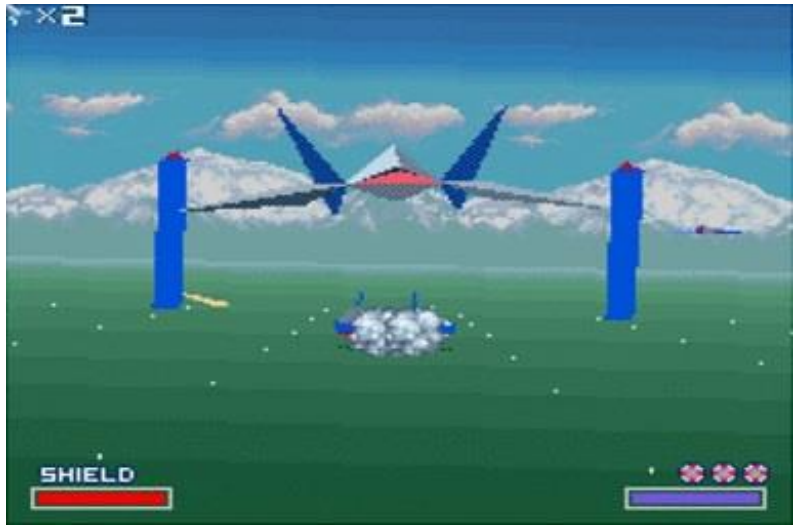
1995

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- **Shigeru Miyamoto (Super Mario developer)** really wanted a 3D flight simulator though
- Worked with a British software company to develop…

3D graphics in consoles (1993)

- Super Nintendo was not powerful enough to draw 3D graphics
- Shigeru Miyamoto (Super Mario developer) really wanted a 3D flight simulator though
- Worked with a British software company to develop…





https://en.wikipedia.org/wiki/Star_Fox_(1993_video_game)

3D graphics in consoles (1993)

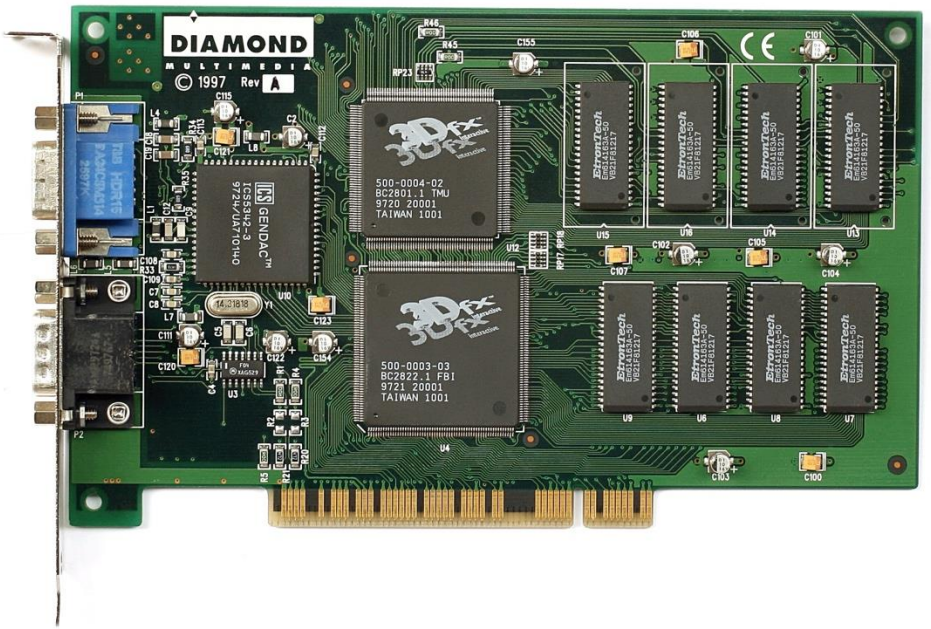
- Game **cartridges shipped with a “mini GPU”** on them:
  - the Super FX



https://twitter.com/gameminesocials/status/1322946537077526528?s=20

3D graphics acceleration

- 1996 : First **3D graphics accelerator**: Voodoo from **3Dfx**
  - Discrete GPU (separate from CPU)
  - Early **3D games: e.g. Tomb Raider**
  - Acquired by Nvidia in 2002



https://en.wikipedia.org/wiki/3dfx_Interactive

3D graphics acceleration

- 3D accelerators continued, many companies **competing**:
  - Nvidia
  - ATI (acquired by AMD)
  - 3Dfx
  - and more...

- Next milestone in 1999:
  - **Nvidia coins the term “GPU”**
  - Compare with modern website

https://web.archive.org/web/20030814003456/www.nvidia.com/object/gpu.html

Programmable 3D accelerators

- 2001: Microsoft **DirectX** 8 required programmable **vertex and pixel shaders**.

- 2001: First GPU to satisfy the requirement was Nvidia GeForce 3
  - Used on the original Xbox

- Programmers **started writing general programs** for these GPUs:
  - Present your data as a graphical input (e.g. Textures and Triangles)
  - Read the output after a series of “graphics” API calls

GPGPU Programming (General Purpose)

- 2006: Nvidia releases **CUDA**: programming language for their GPUs
  - Supported by GeForce 8 series.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point

- Soon after...

GPGPU Programming

- 2006: Nvidia releases CUDA: programming language for their GPUs
  - Supported by 8th generation CUDA devices.
  - Integrated vertex and pixel cores into “shader cores”
  - Support for IEEE floating point

- Soon after...

- 2008: The **Khronos** Group launches **OpenCL for cross vendor GPGPU**:
  - including AMD, Intel, Qualcomm

Khronos Group



- Started in 2000 **by Apple** as a standards body for graphics API to **counter Microsoft's DirectX**:
  - A way to unify APIs across many different vendors
  - At the time: ATI, Nvidia, Intel, Sun Microsystems (and a few others)
  - Now: Many companies, including AMD, Nvidia, Intel, Qualcomm, ARM, Google

  - **OpenGL** is maybe the biggest standard they maintain (**for graphics**)
  - **OpenCL** is big **for compute**
  - **Vulkan** unifies **compute + graphics** (mostly graphics)
- **Apple deprecated Khronos** group standards to support **Metal** in 2018
  - iPhone used to have **GPUs from a company called Imagination, a British company**. Apple took their engineers.
- Our laptops may have GPUs from **Intel or AMD**. Now apple laptops has **apple GPUs**.

https://en.wikipedia.org/wiki/Khronos_Group

Where are we now?

- **Nvidia CUDA** is widely used, driving many HPC and ML applications
  - Nvidia heavily invested in its software stack and made it accessible, and this made it widely used.
  - Nvidia GPUs are **not designed power efficient**, and are **not used much in mobile devices**.
- **OpenCL** is used to program **other GPUs** (although it is not as widely used)
- **Metal** is used for **Apple devices**
- **Vulkan has momentum**, but mostly in graphics

- New GPGPU programming languages are on the horizon:
  - **WebGPU** - **a javascript interface to unite Metal, Vulkan and DirectX**
  - It's ambitious! Ships in Chrome, Firefox and Safari.

GPU Shortages?

- **Cryptocurrency**:
  - 2018 reported tripling of GPU prices and shortages due to increase demand from miners.
  - Still happening with lots of market fluctuations.
  - Still plenty of GPUs in your phone, laptop, etc. ☺

GPUs now

- **Nvidia is #2 most valuable company** (after Apple) in the world at roughly $4.94 trillion,
  - More valuable than Microsoft, Alphabet and Amazon
- Graphics are still a valuable domain, but **AI** seems to be more valuable
  - Valuable to learn general purpose GPU programming (GPGPU)
- **Many companies** make their own GPUs:
  - AMD, Apple, Qualcomm, ARM, Intel, Imagination, Broadcom
- **Many startups** in the Valley making new AI chips similar to GPUs

Teaching GPU programming

- This is difficult!
- Nvidia GPUs have the most straightforward programming model (**CUDA**).
- It is extremely difficult to get a class of students access to Nvidia GPUs these days.
  - AWS? Expensive and often oversubscribed w.r.t. GPUs
  - Department? ML folks get priority and super computing clusters are painful

Plan

- The GPU programming lectures will use **CUDA**
  - It is widely used
  - The programming model is straightforward

- Homework will use **WebGPU**, because it is widely supported
  - *There are more non-Nvidia GPUs in this room than Nvidia GPUs*

Plan

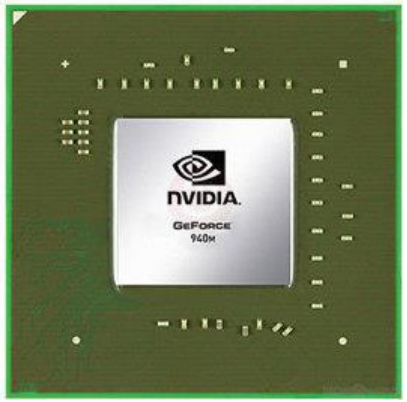
- The homework uses Javascript as its ”CPU” language, and WebGPU as its ”GPU” language.

- We have provided generous skeletons for the homework. We can go over some javascript, but it is a high-level language and should not be hard to figure out what you need to do.

- The WebGPU portion is straightforward, and I will provide a mapping directly from what we talked about to what you need.

Programming a GPU

Tiny GPU in an embedded system





The CPU



Nvidia Jetson Nano (whole chip, CPU + GPU)
2 Billion transistors
10 TDP (Thermal Design Power)
Est. $99

similar transistor count

Intel i7-9700K
2.16 Billion transistors
95 TDP
Est. $316

https://www.techpowerup.com/gpu-specs/geforce-940m.c2648
https://www.alibaba.com/product-detail/Intel-Core-i7-9700K-8-Cores_62512430487.html
https://www.prolast.com/prolast-elevated-boxing-rings-22-x-22/

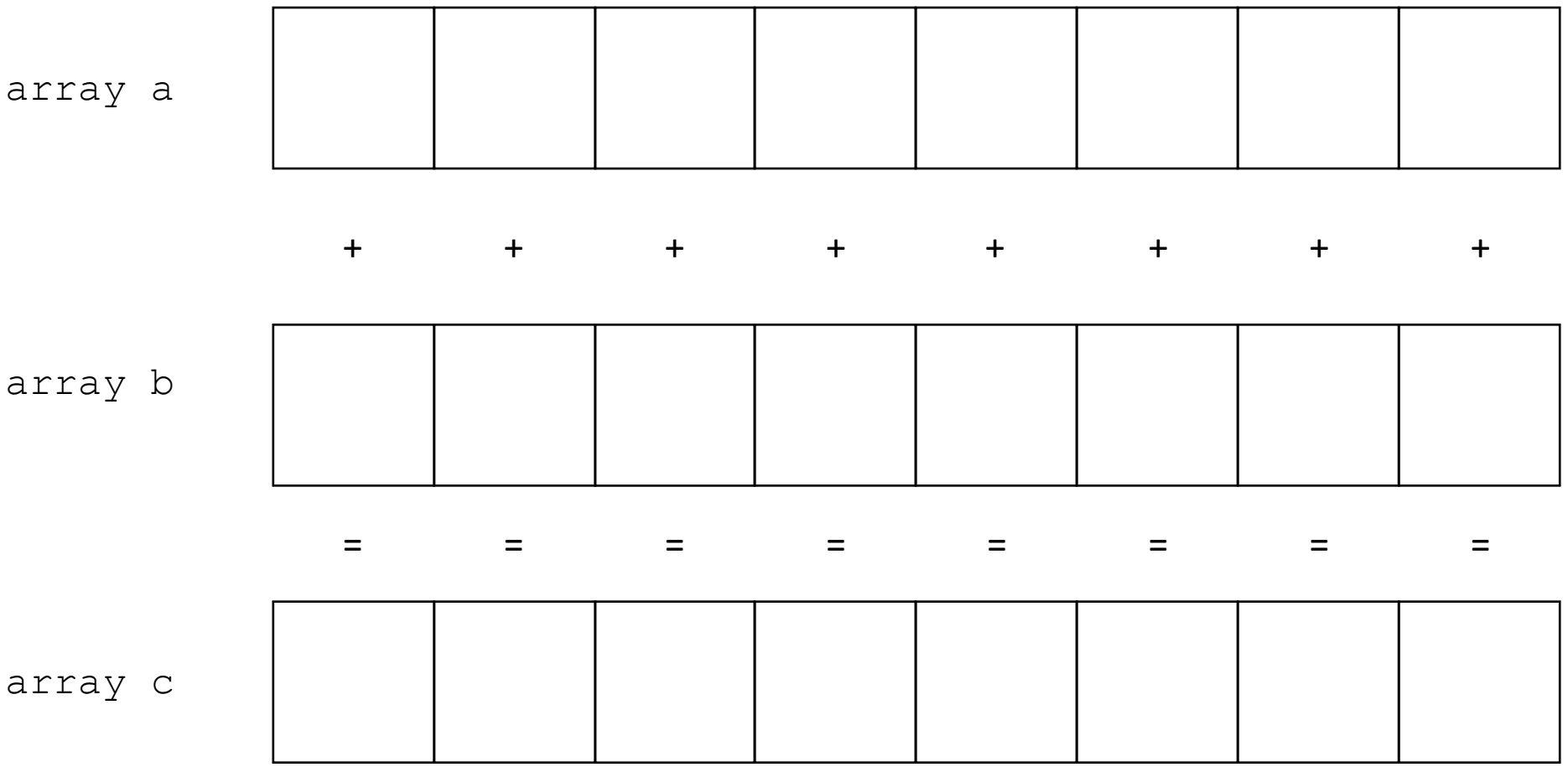
Programming a GPU

- The problem: Vector addition

Embarrassingly parallel

Computation can easily be divided into threads

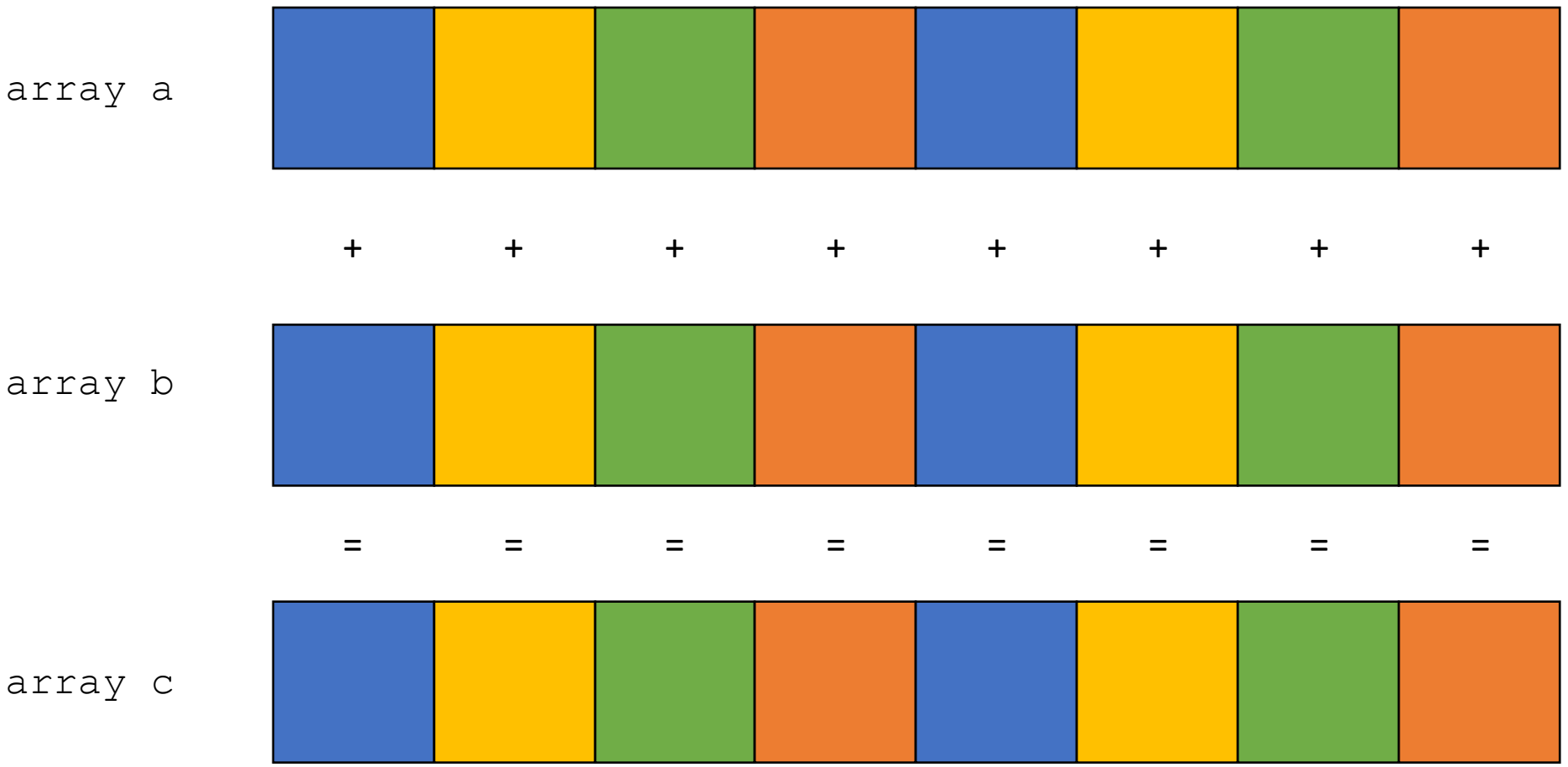
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

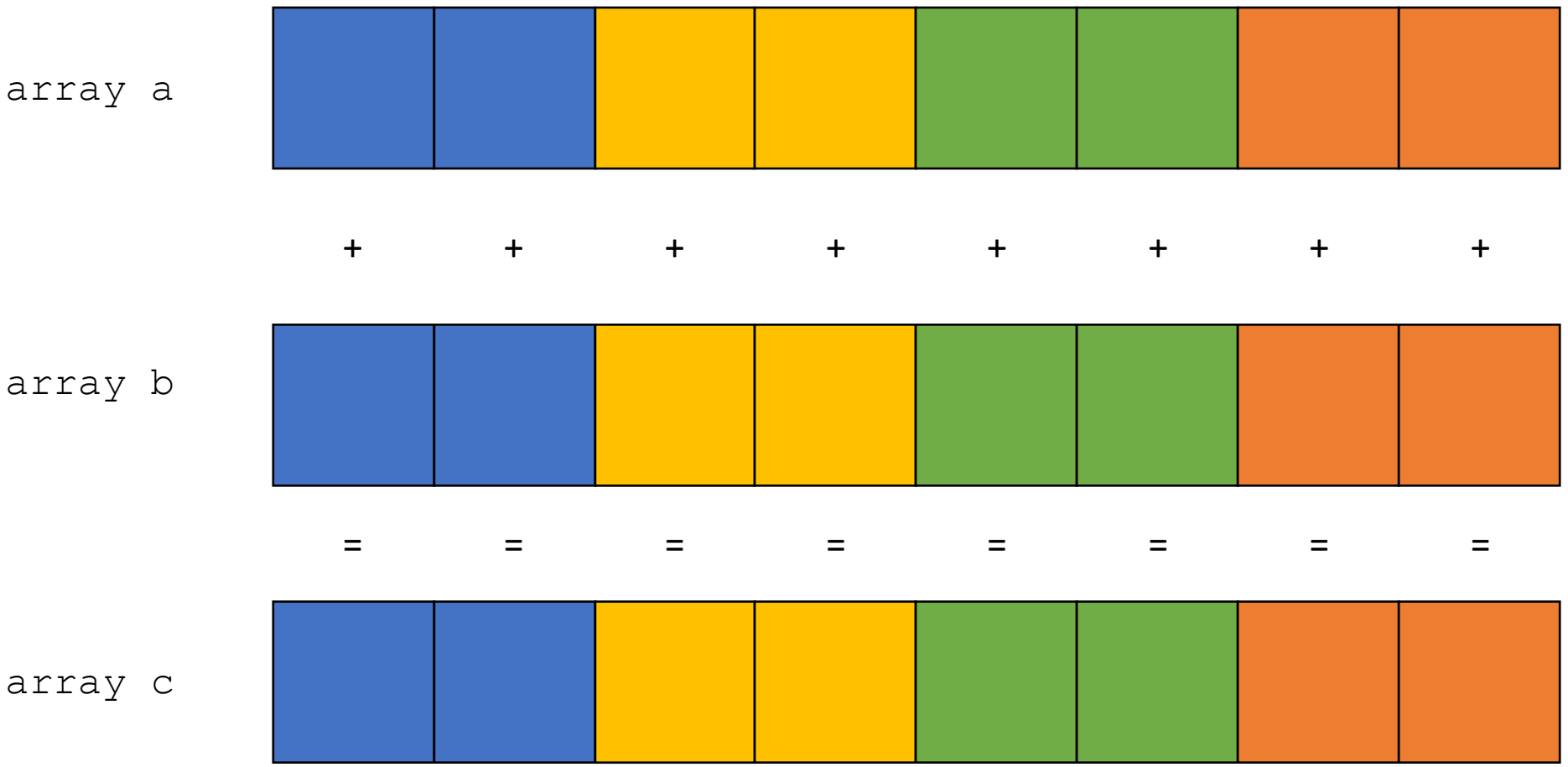
Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Parallel Schedules

Computation can easily be divided into threads

Thread 0 - Blue
Thread 1 - Yellow
Thread 2 - Green
Thread 3 - Orange



Programming a GPU

- The problem: Vector addition

- Who can do it faster?

Let’s set up the CPU

```
void vec_add(double *a, double *b, double *c, int size,
int tid, int num_threads) {
  int chunk = size / num_threads;
  int start = chunk * tid;
  int end = start + chunk;
  for (int j = 0; j < OUTER; j++) {
    for (int i = start; i < end; i++) {
      a[i] = b[i] + c[i];
    }
  }
}
```

```
for (int i = 0; i < num_threads; i++)
  thread_array[i] = thread(vec_add, a, b, c, SIZE, i,
num_threads);

for (int i = 0; i < num_threads; i++)
  thread_array[i].join();
```

Now for the GPU

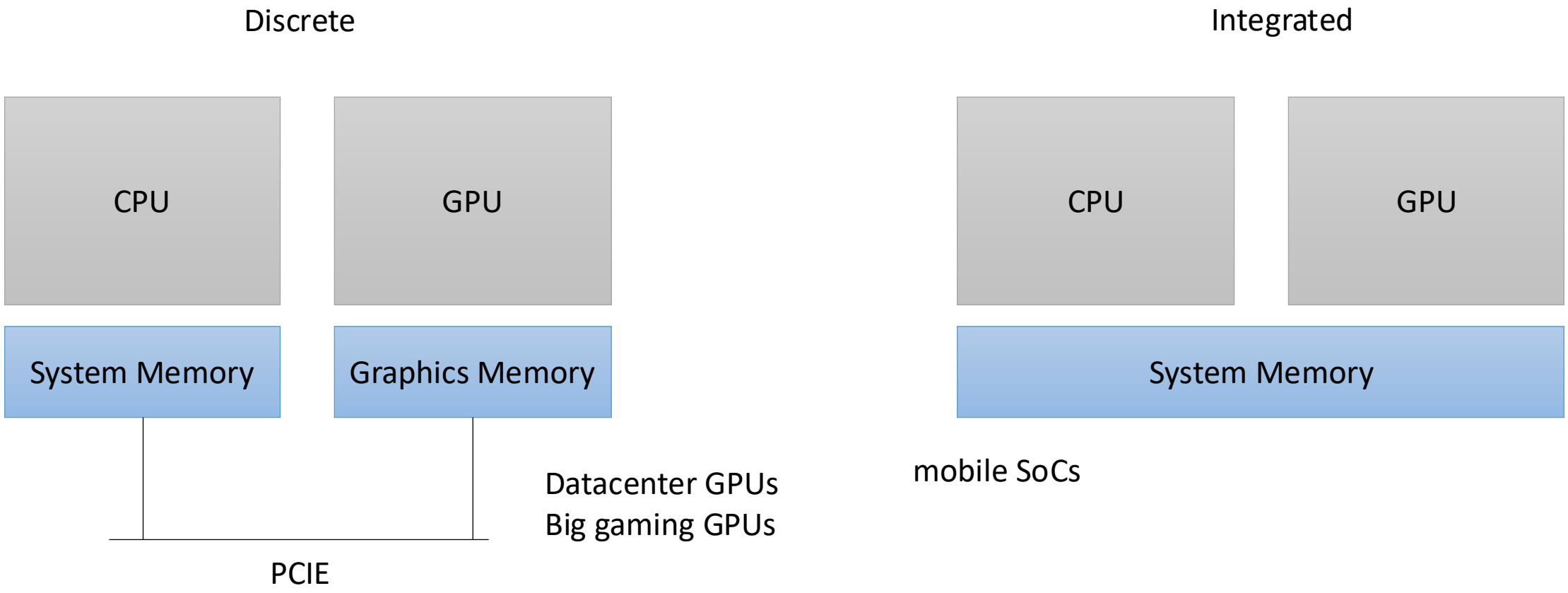
- It’s going to take a bit of work....

GPU set up

- We need to allocate and initialize memory

GPU set up

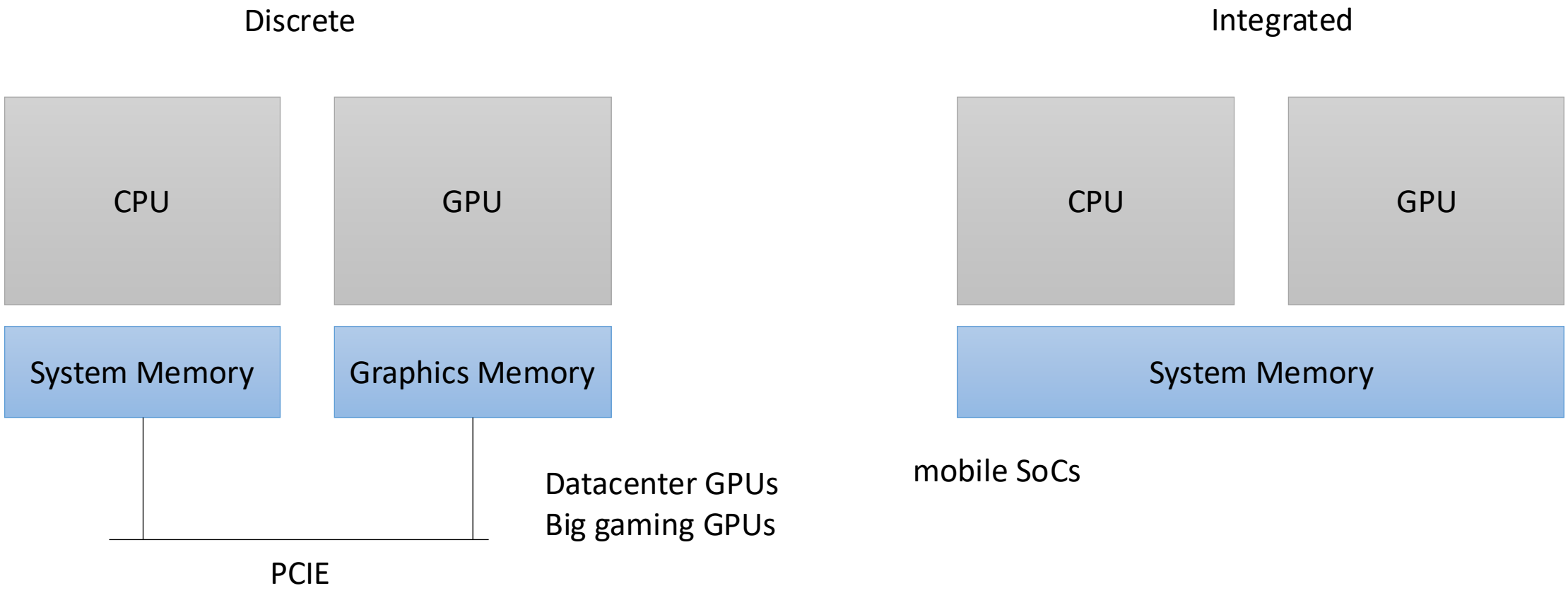
- GPUs come in two flavors



GPU set up

Pros and cons of each?

- GPUs come in two flavors

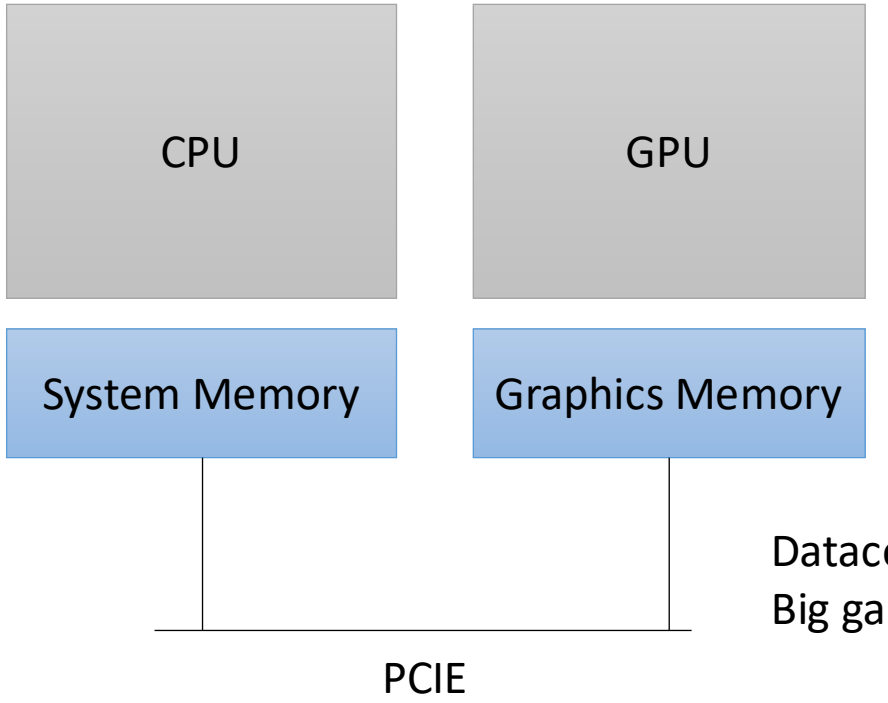


GPU set up

- GPUs come in two flavors

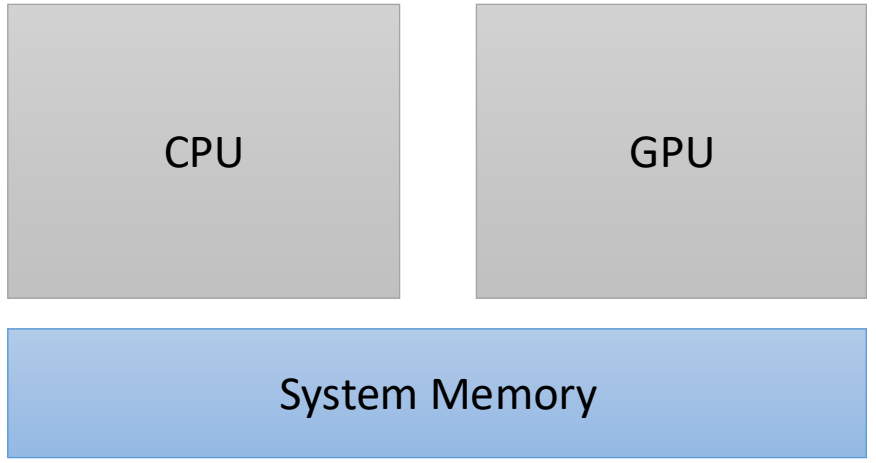
Pros and cons of each?
*Different types of memory for discrete
*Swappable for discrete
*More energy efficient for integrated
*Better memory utilization for integrated

Discrete



Datacenter GPUs
Big gaming GPUs

Integrated



mobile SoCs

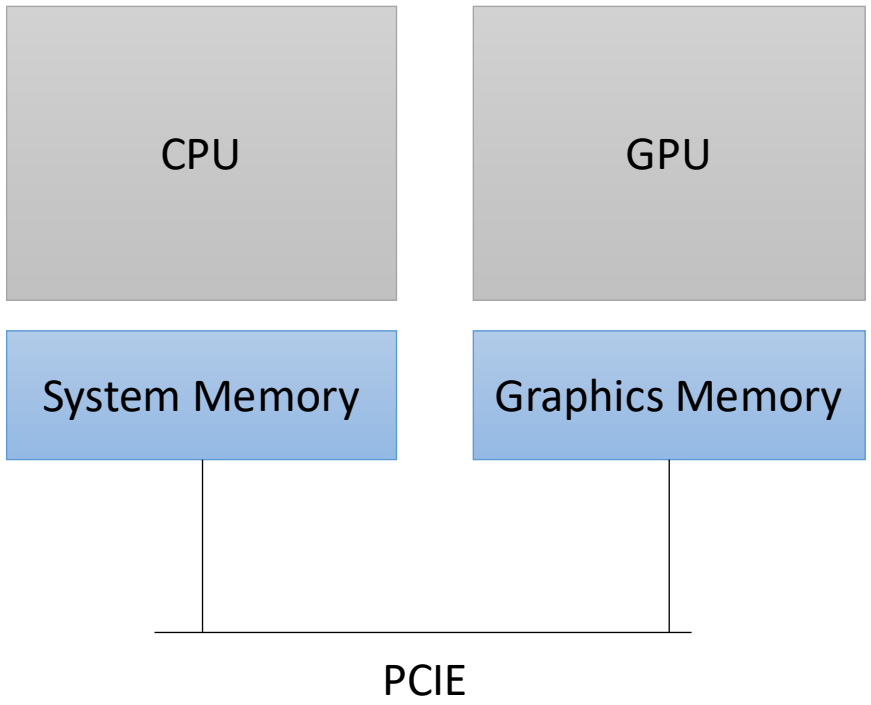
GPU set up

- GPUs come in two flavors

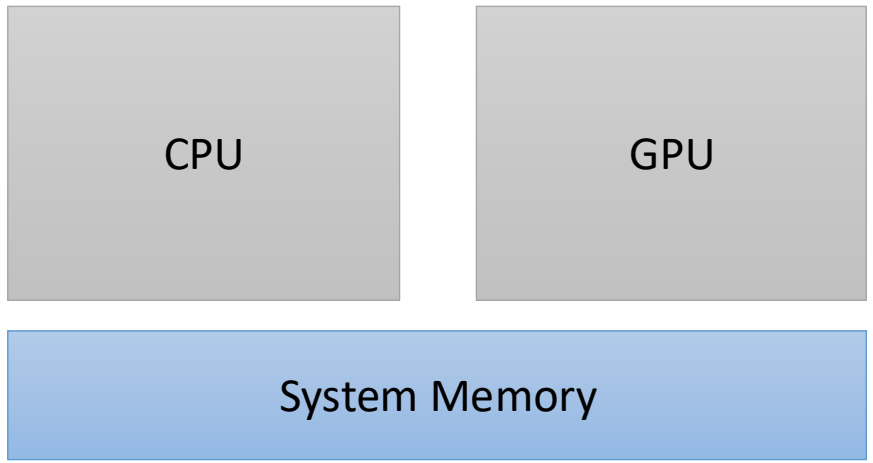
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



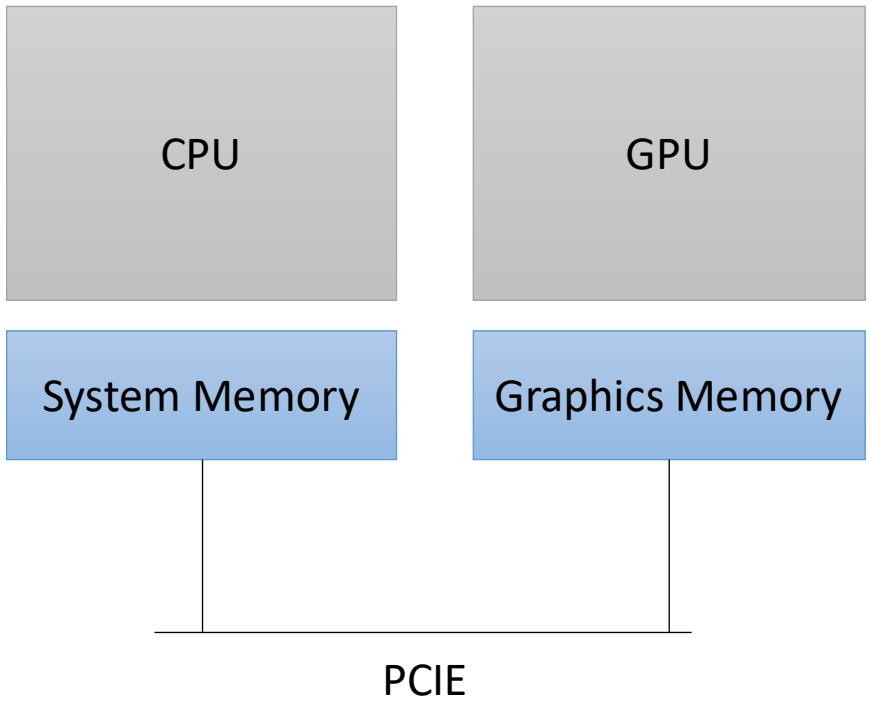
GPU set up

- GPUs come in two flavors

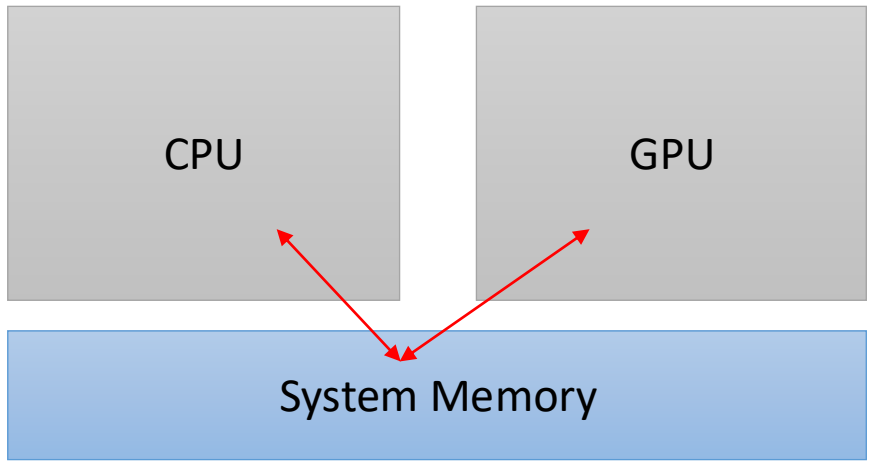
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



Integrated



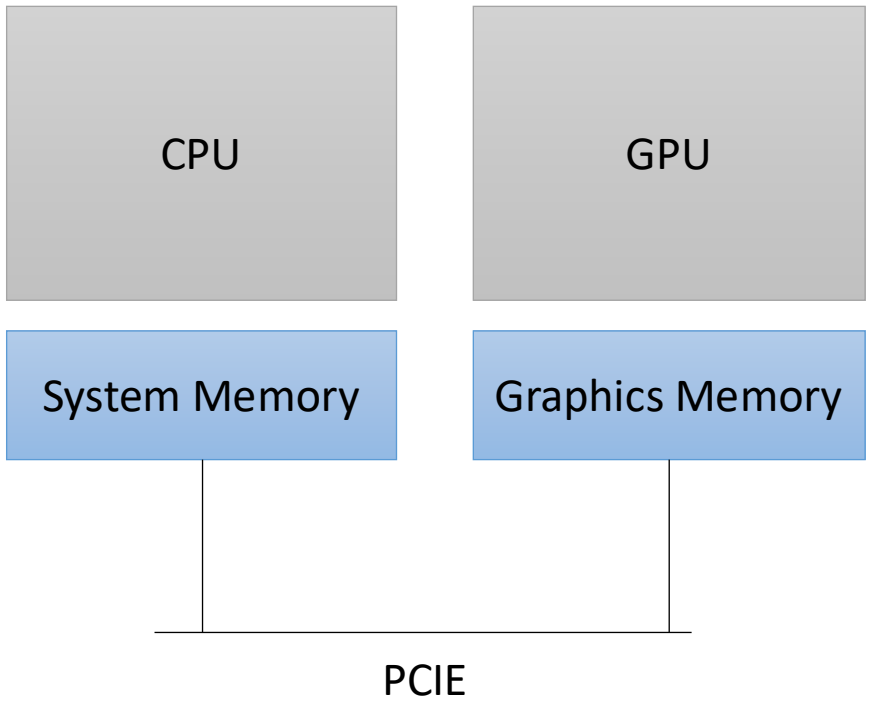
GPU set up

- GPUs come in two flavors

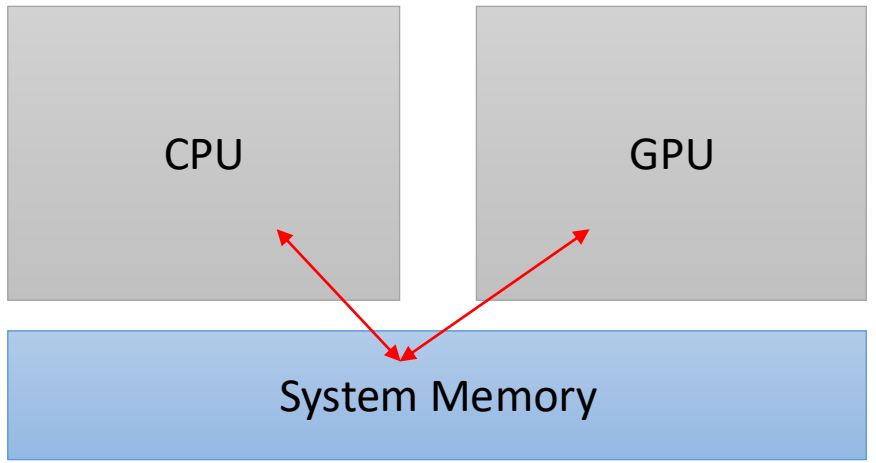
Although mobile GPUs share the system memory, Most still require you to program as if they didn't have shared memory.

Why?

Discrete



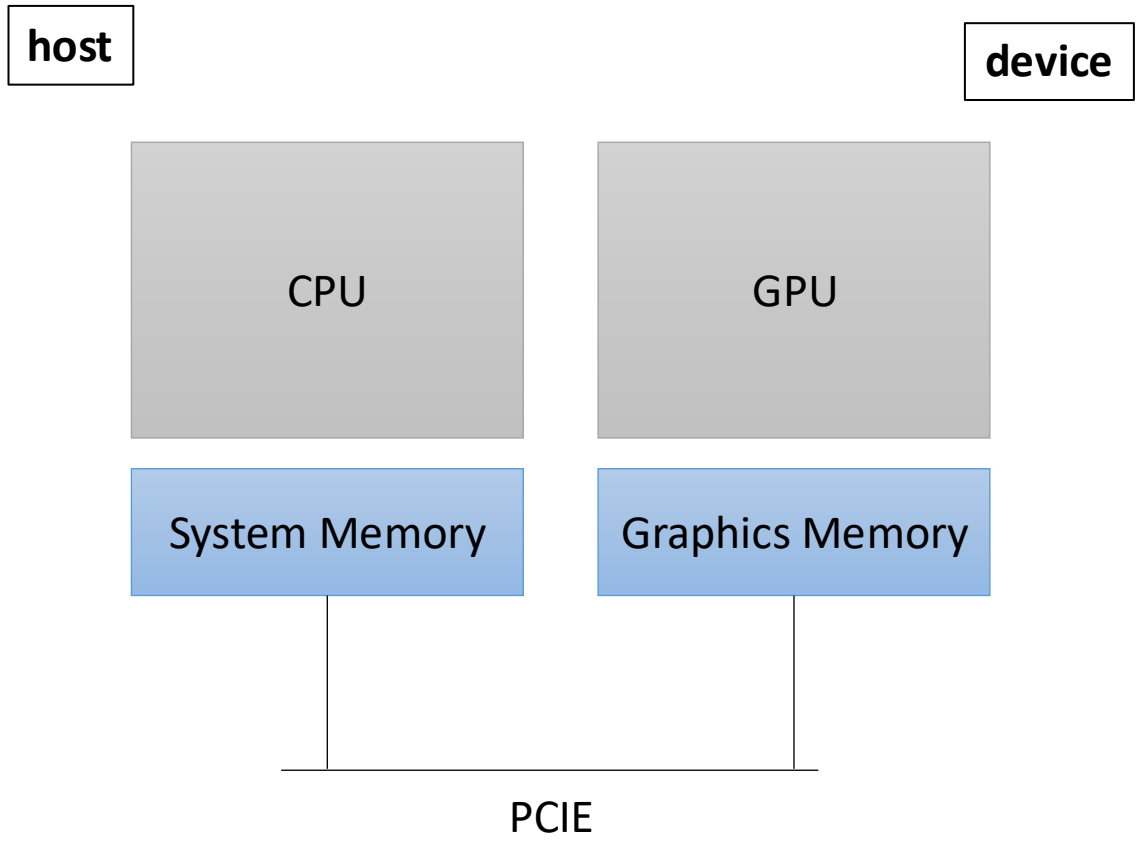
Integrated



In many cases, CPU-GPU communication is not fully supported. Coherence, fences, and RMWs might not be supported.

GPU set up

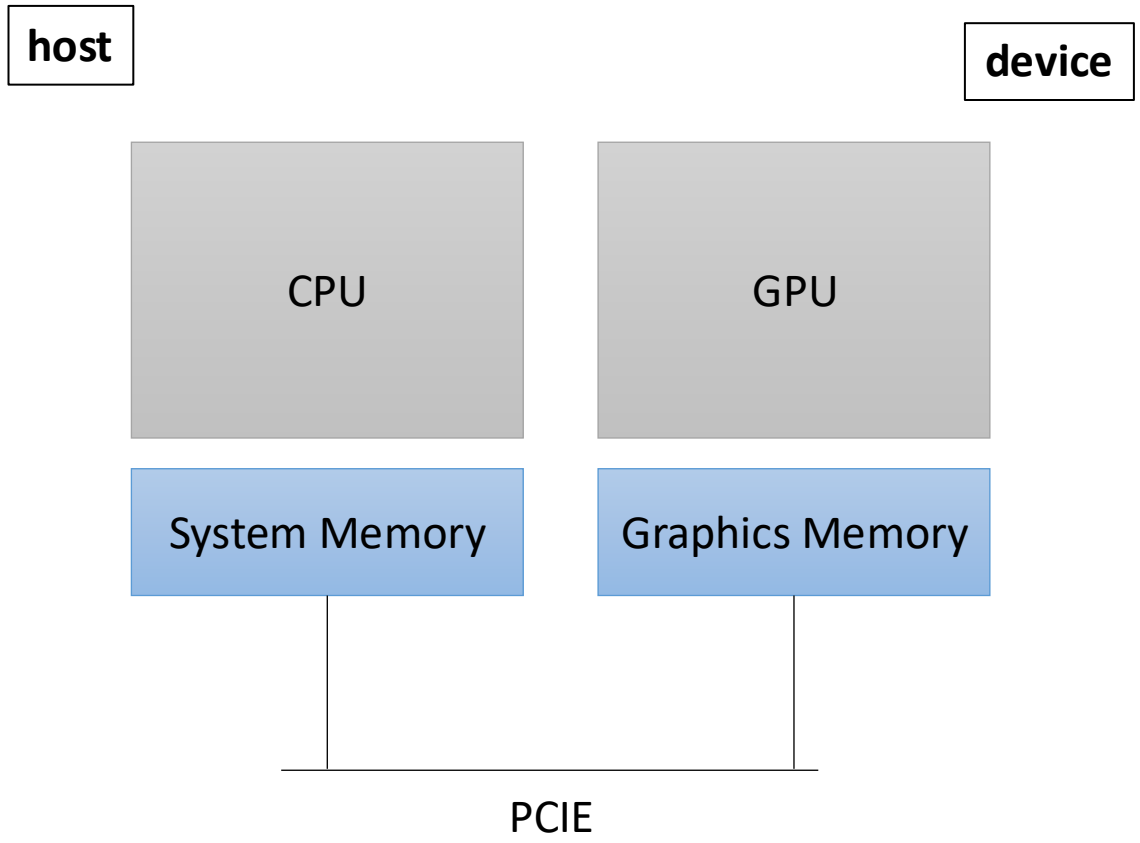
- Our heterogeneous, parallel, programming model



GPU set up

- Our heterogeneous, parallel, programming model

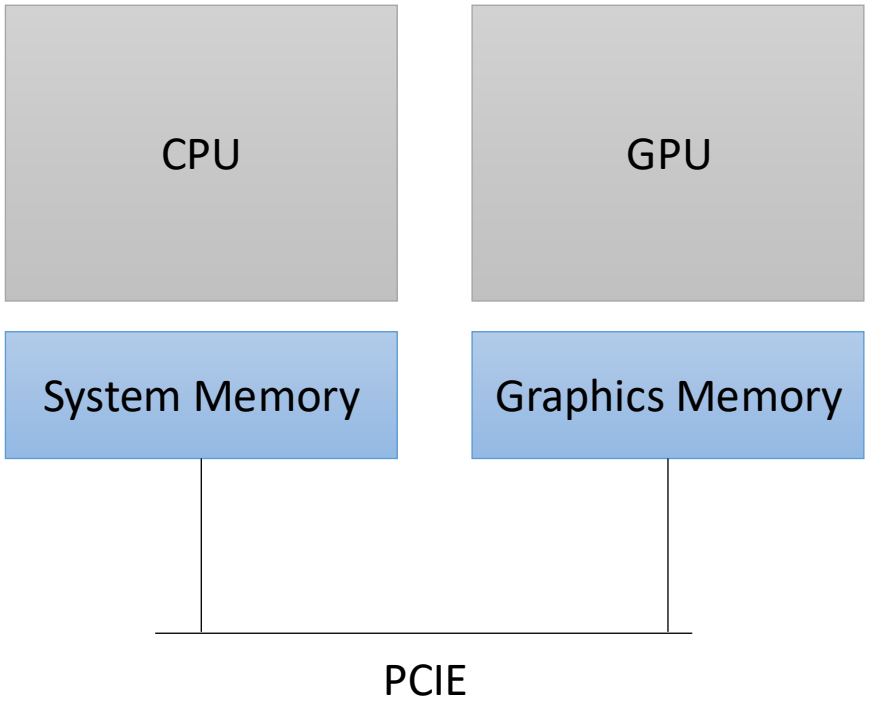
The host (CPU) will write a C++-like program that allocates and sets up memory on the GPU. The host will then call a GPU program called a kernel.



GPU set up

How do we allocate memory on a CPU?

- Our heterogeneous, parallel, programming model

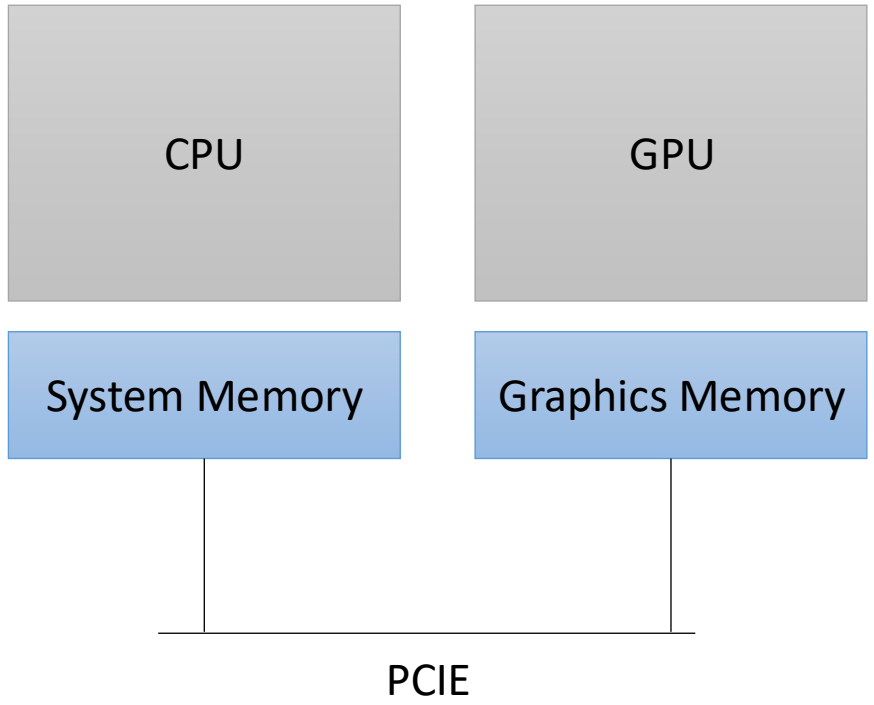


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

```
int *x = (int*) malloc(sizeof(int)*SIZE);
```

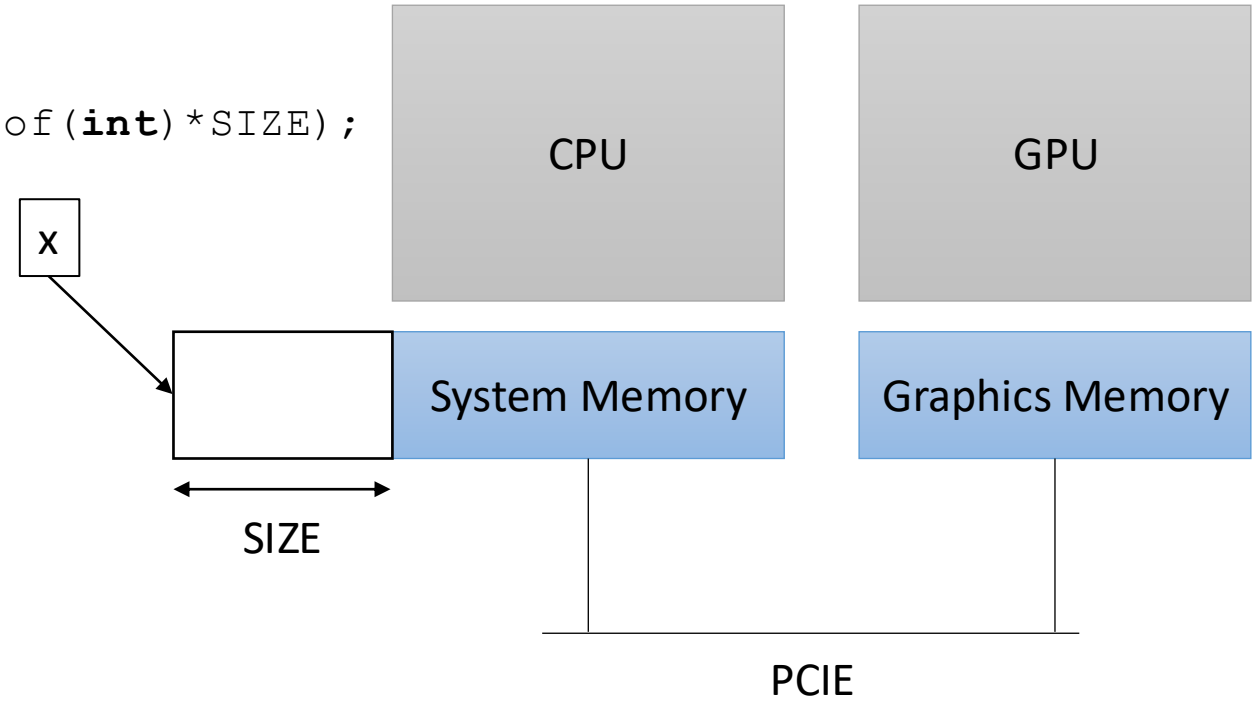


GPU set up

How do we allocate CPU memory on the host?

- Our heterogeneous, parallel, programming model

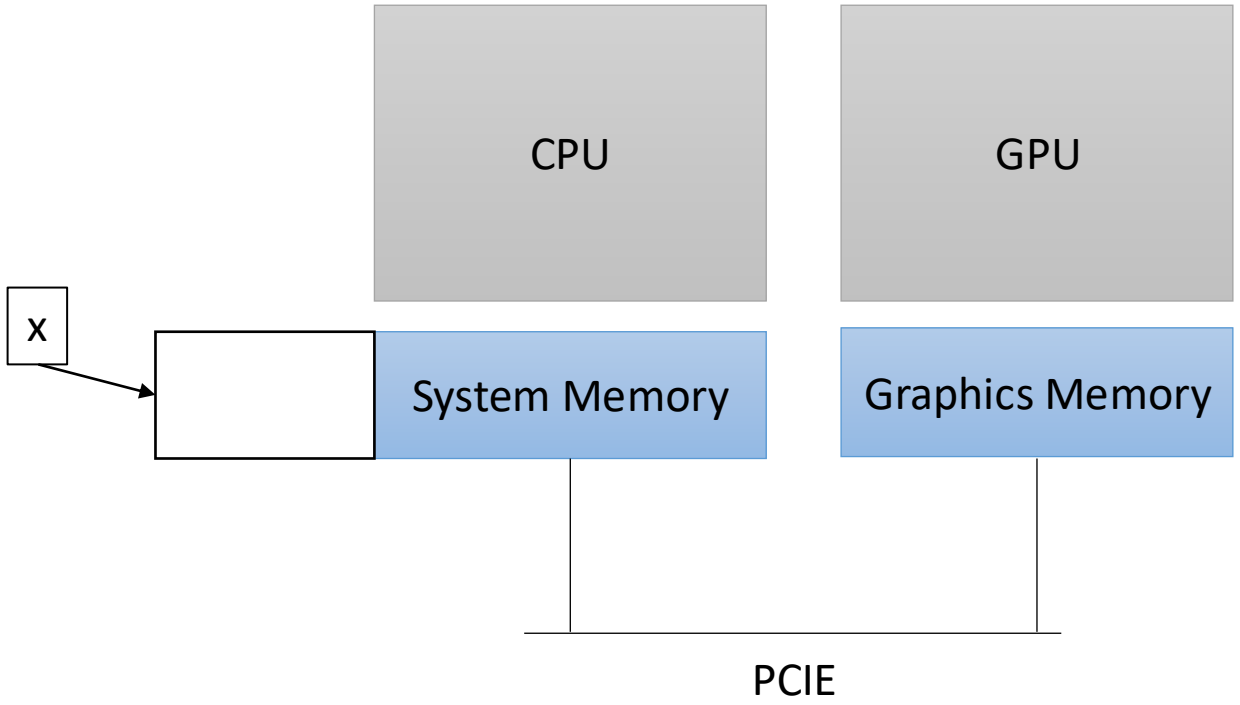
```
int *x = (int*) malloc(sizeof(int)*SIZE);
```



GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

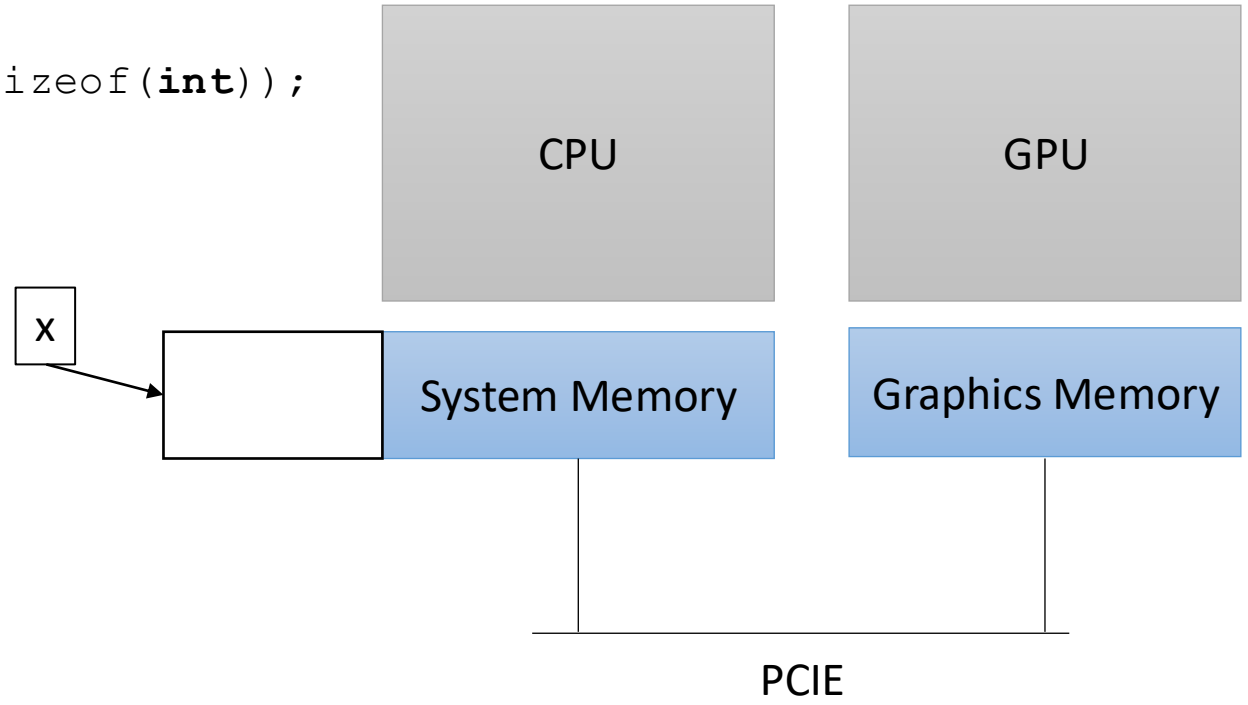


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

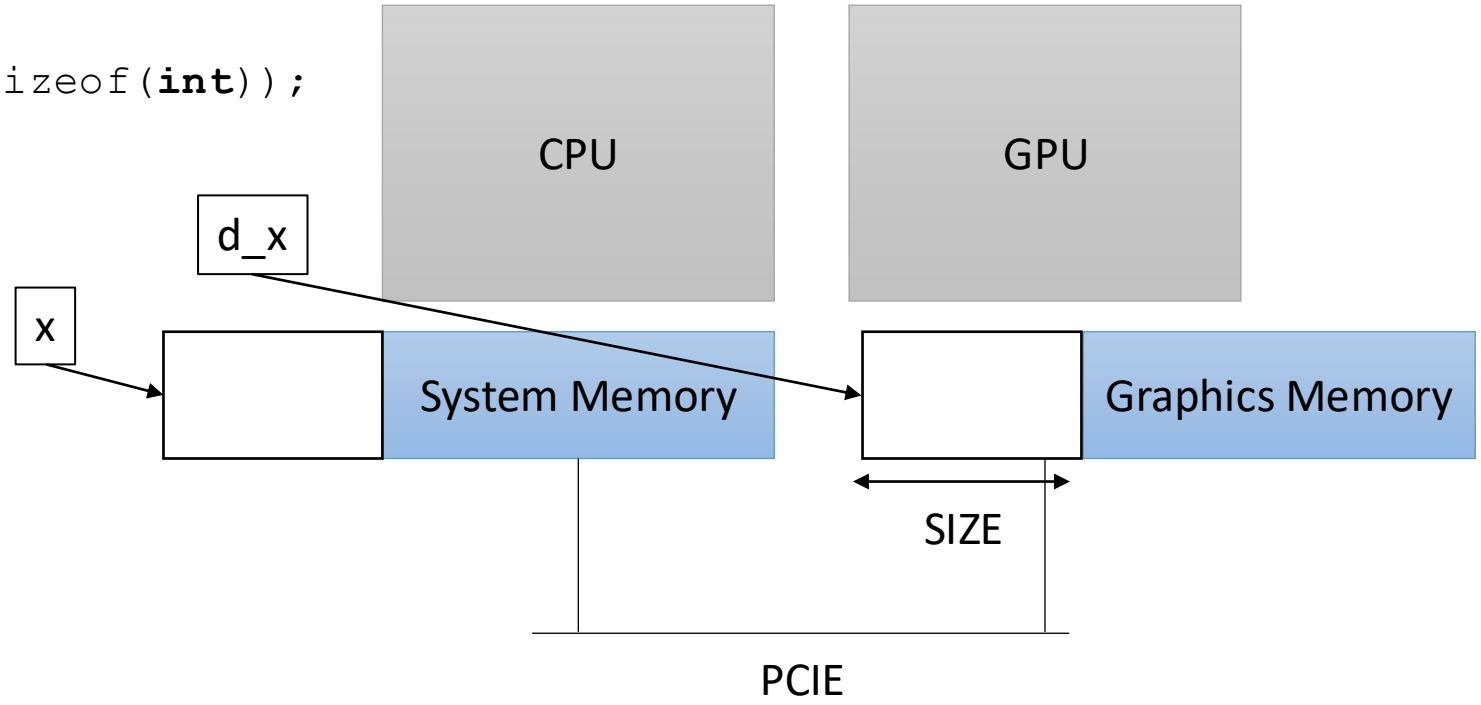


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

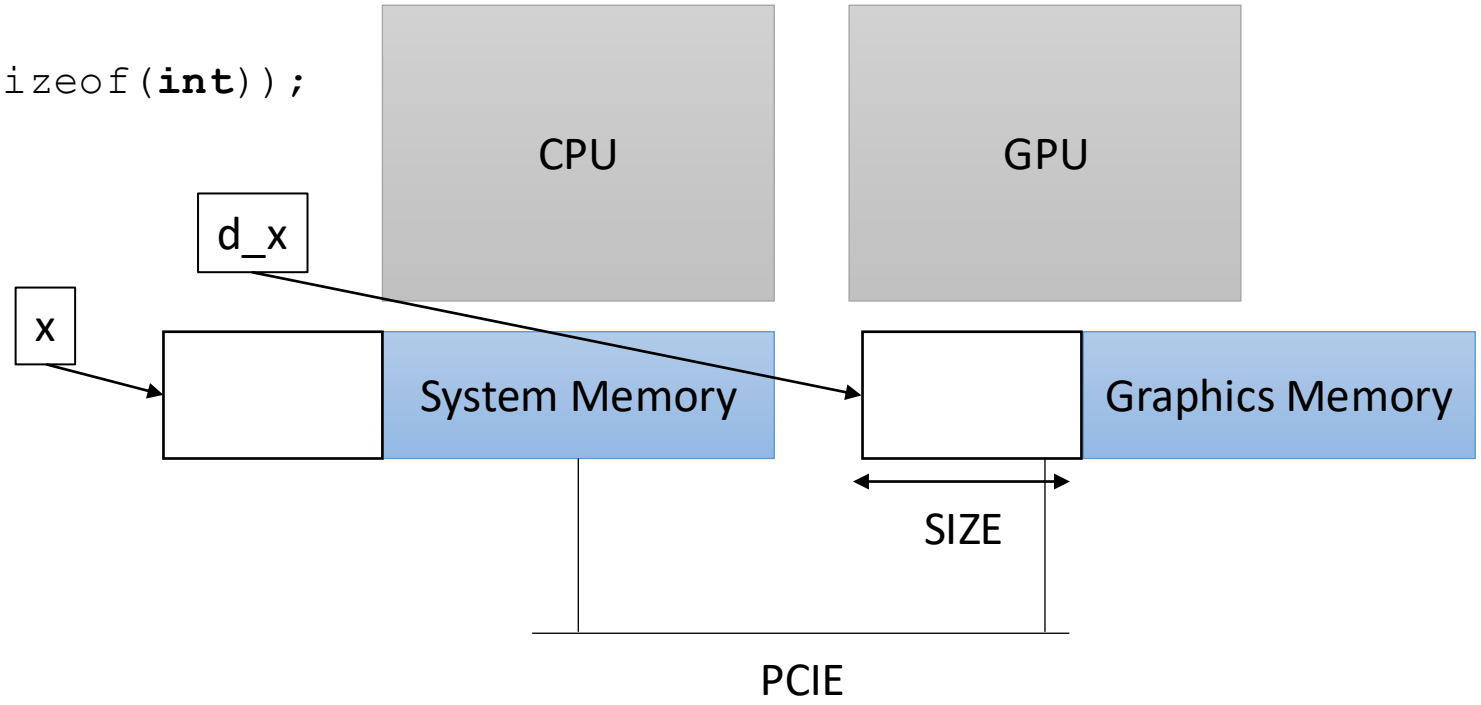


GPU set up

We need to allocate GPU memory on the host

- Our heterogeneous, parallel, programming model

```
int *d_x;
cudaMalloc(&d_x, SIZE*sizeof(int));
```

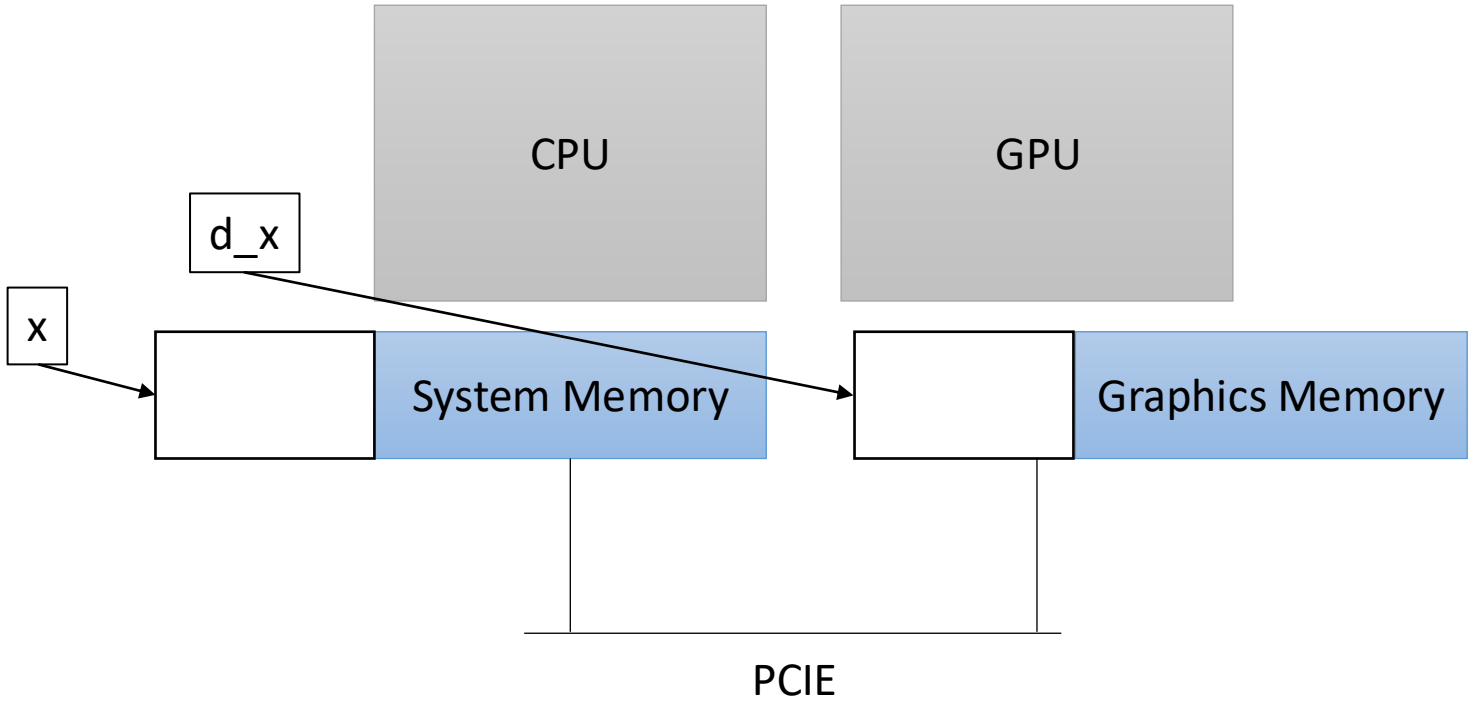


`d_x` is a pointer, in the CPU program, that points to memory on the GPU.

We can pass the pointer around, but the CPU cannot access the data i.e. `d_x[0]` gives an error!

GPU set up

- Our heterogeneous, parallel, programming model

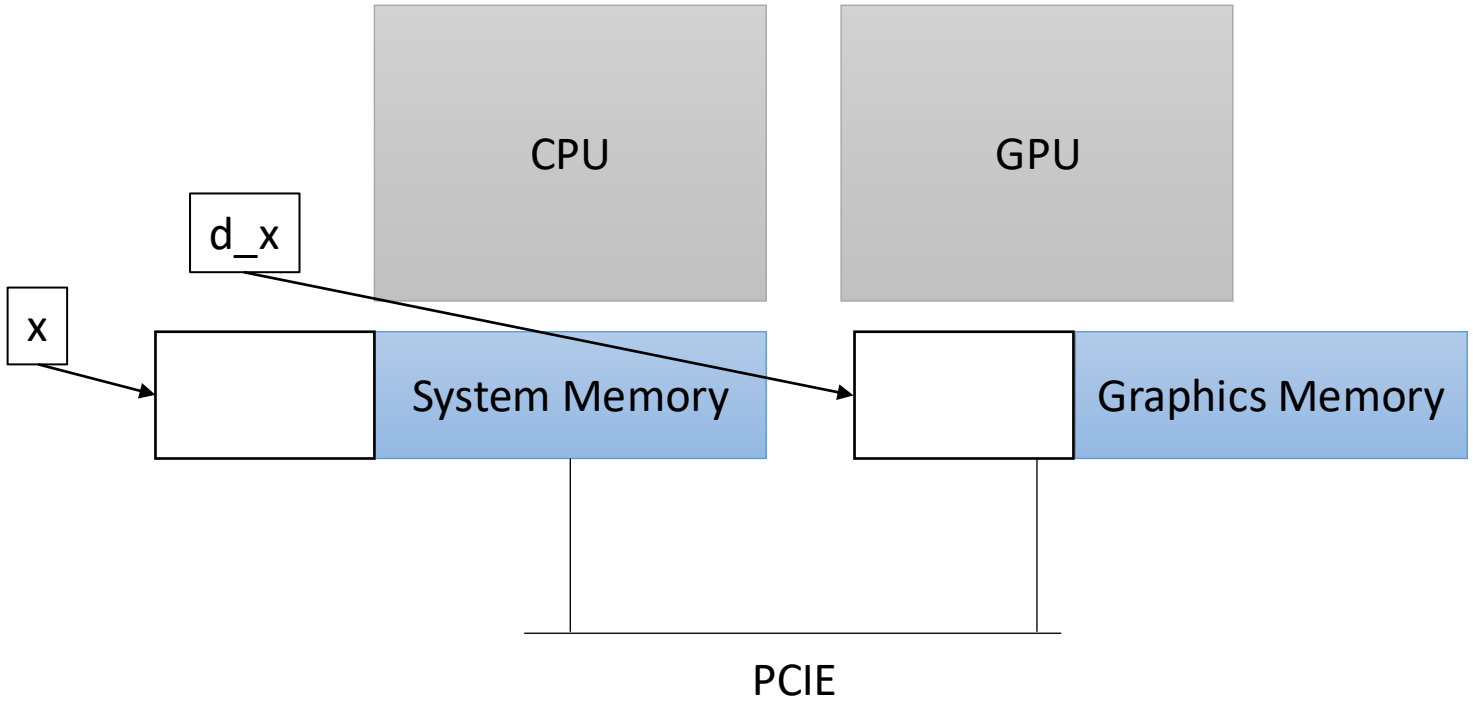


GPU set up

- Our heterogeneous, parallel, programming model

```
cudaFree(d_x)
```

device memory is scarce, and a program that allocates inside a loop without freeing runs out of GPU memory quickly.

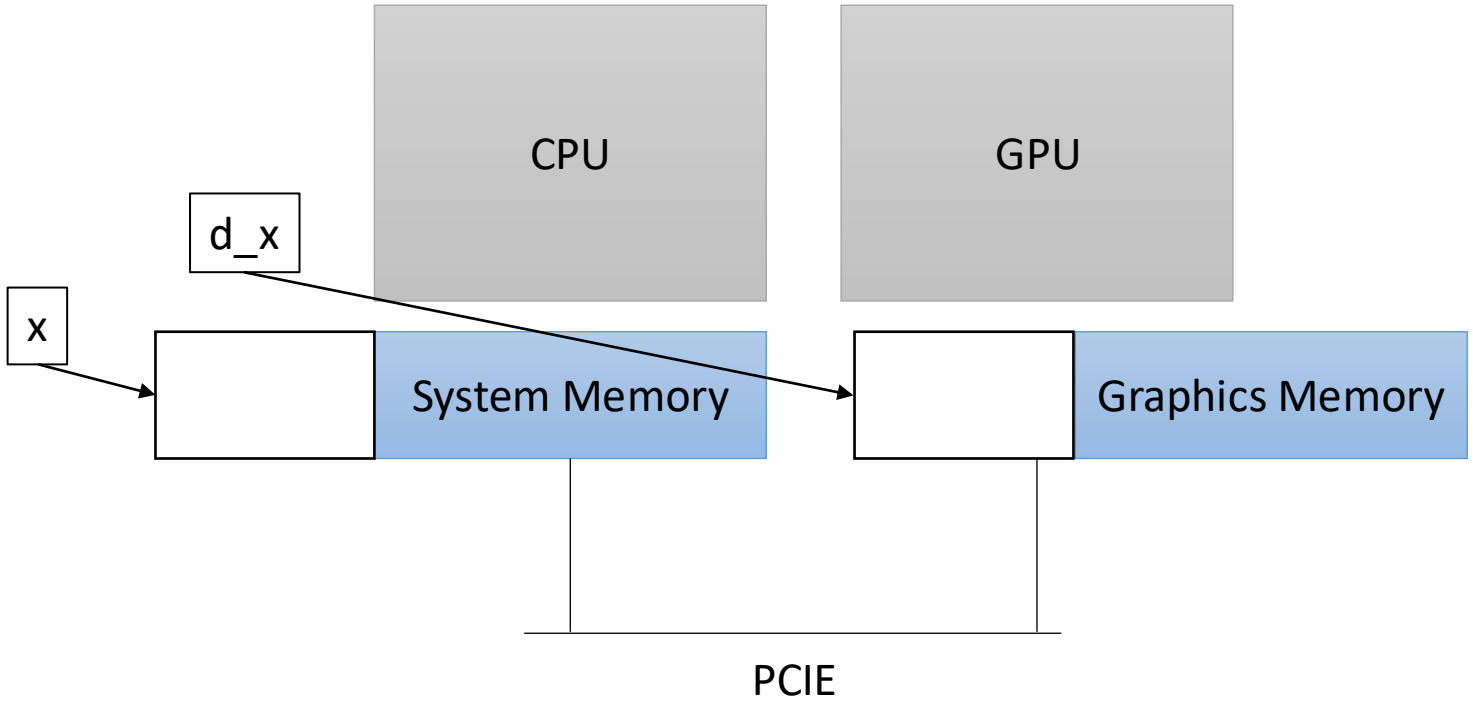


GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk



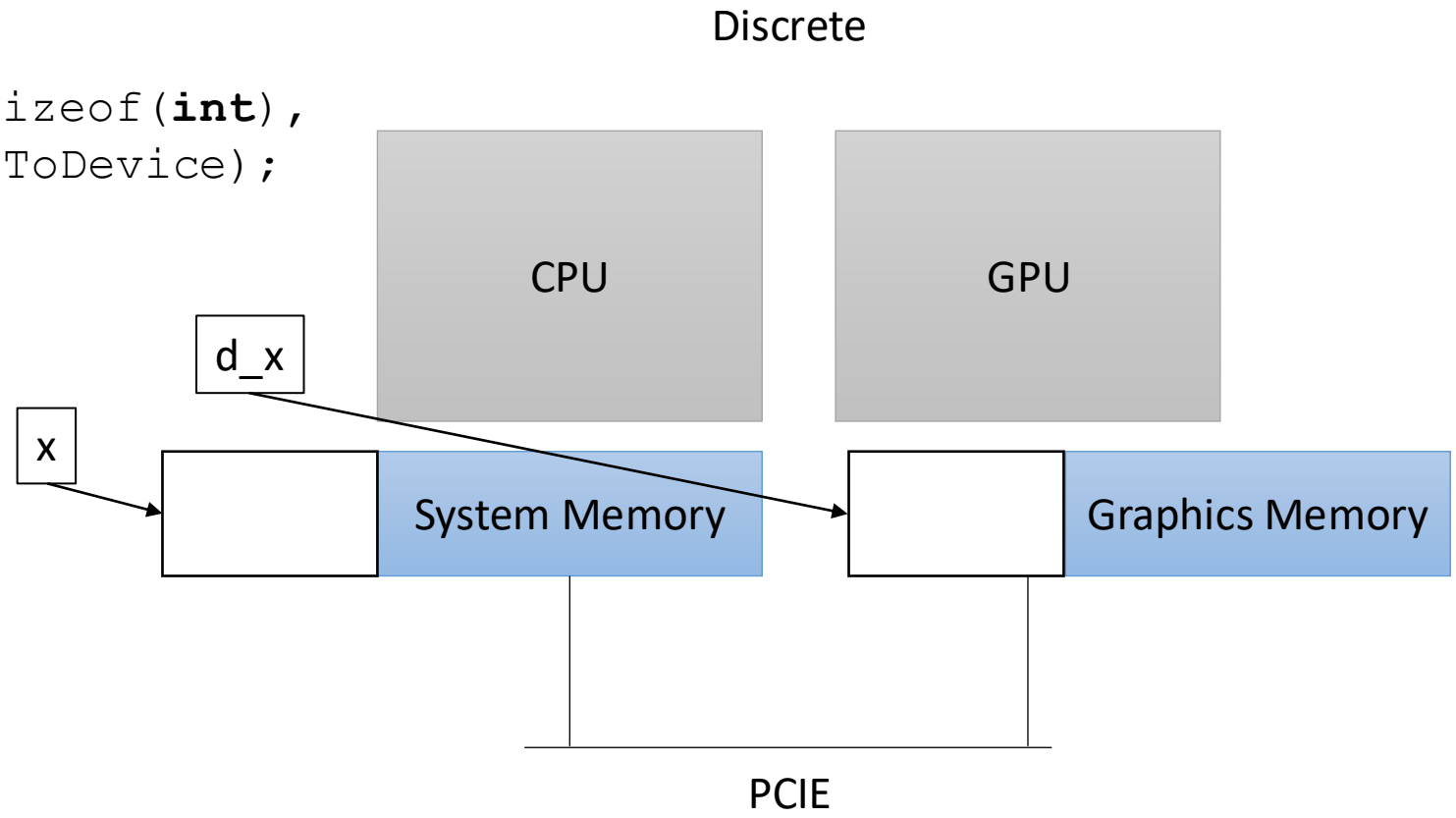
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



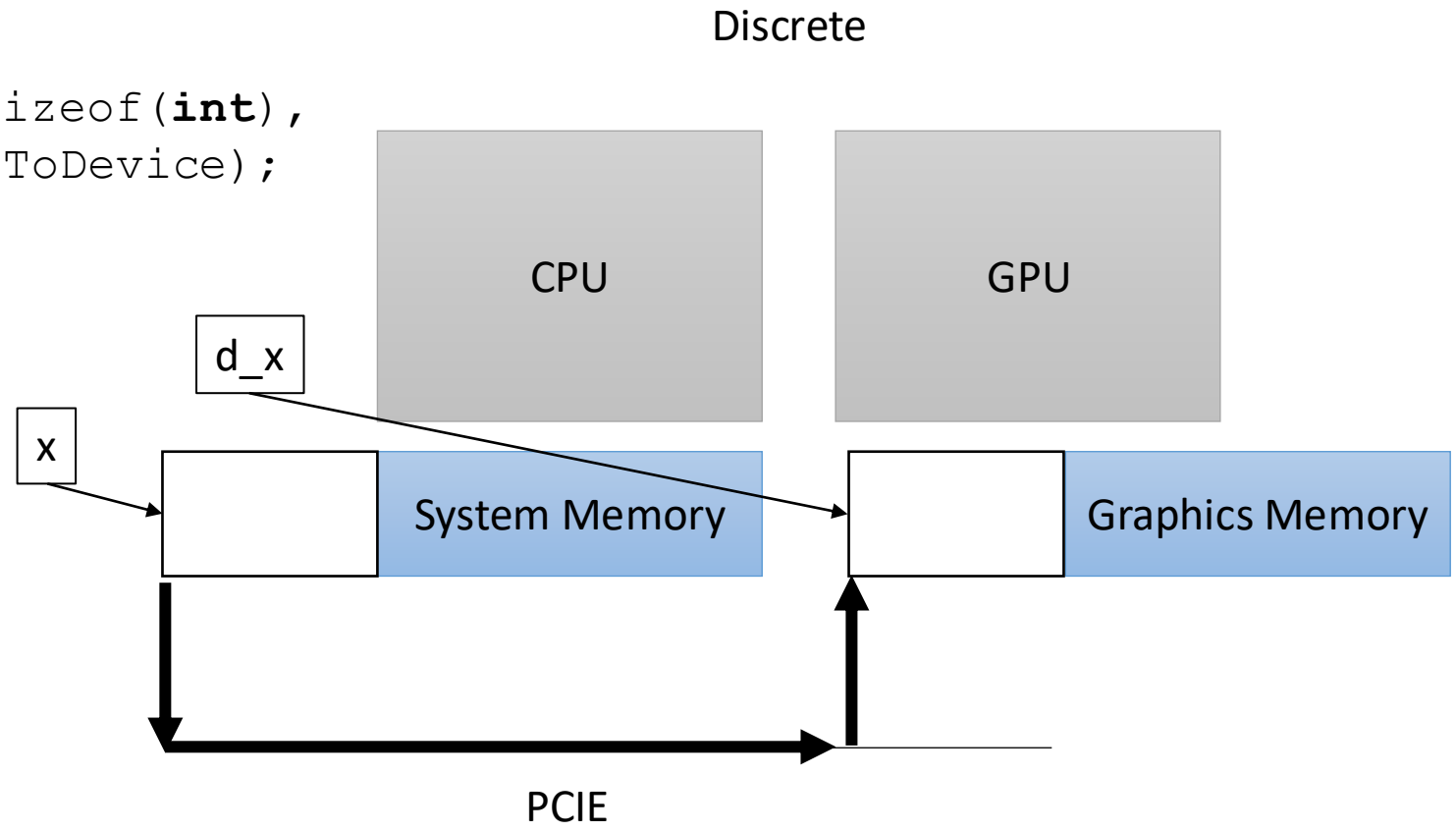
GPU set up

- Our heterogeneous, parallel, programming model

If we can't access d_x on the CPU, how do we initialize the memory?

GPU has no access to input devices e.g. disk

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



How does this look in code?

```
float *a = new float[SIZE];
float *b = new float[SIZE];
float *c = new float[SIZE];

for (int i = 0; i < SIZE; i++) {
  a[i] = 0.0f;
  b[i] = i;
  c[i] = 1.0f;
}

float *d_a, *d_b, *d_c;
int e = 0;
e = cudaMalloc(&d_a, SIZE*sizeof(float));
e |= cudaMalloc(&d_b, SIZE*sizeof(float));
e |= cudaMalloc(&d_c, SIZE*sizeof(float));

e |= cudaMemcpy(d_a, a, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_b, b, SIZE*sizeof(float), cudaMemcpyHostToDevice);
e |= cudaMemcpy(d_c, c, SIZE*sizeof(float), cudaMemcpyHostToDevice);
```

The GPU Program

- Write a special function in your C++ code.
  - Called a Kernel
  - Use the new keyword `__global__`
  - Keywords in
    - OpenCL `__kernel`
    - Metal `kernel`

- Write it how you'd write any other function

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

Quiz 11.

What keyword do we need to include for a function to be executed on the GPU using CUDA?

Select one

A. __kernel__

B. __global__

C. __this__

D. __gpu__

Scan to answer

forms.gle/9Bi6y8tjrNJW51iw6

chart: docs.google.com/forms/d/e/1FAIpQLSclk6ovyBEa3PJZYObGvoL2y5oHdGaCi2ABDs5X1cmriyDlJQ/viewanalytics

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * a, int * b, int * c, int size) {
    for (int i = 0; i < size; i++) {
      a[i] = b[i] + c[i];
   }
}
```

What in the world?
special new CUDA syntax. We will talk more soon

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

Pass in pointers to memory on the device

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
```

Quiz 11.

The host (CPU) will write a C++-like program that allocates and sets up memory on the GPU. The host will then call a GPU program called a kernel. Is this correct?

Select one

A. True

B. False

Scan to answer

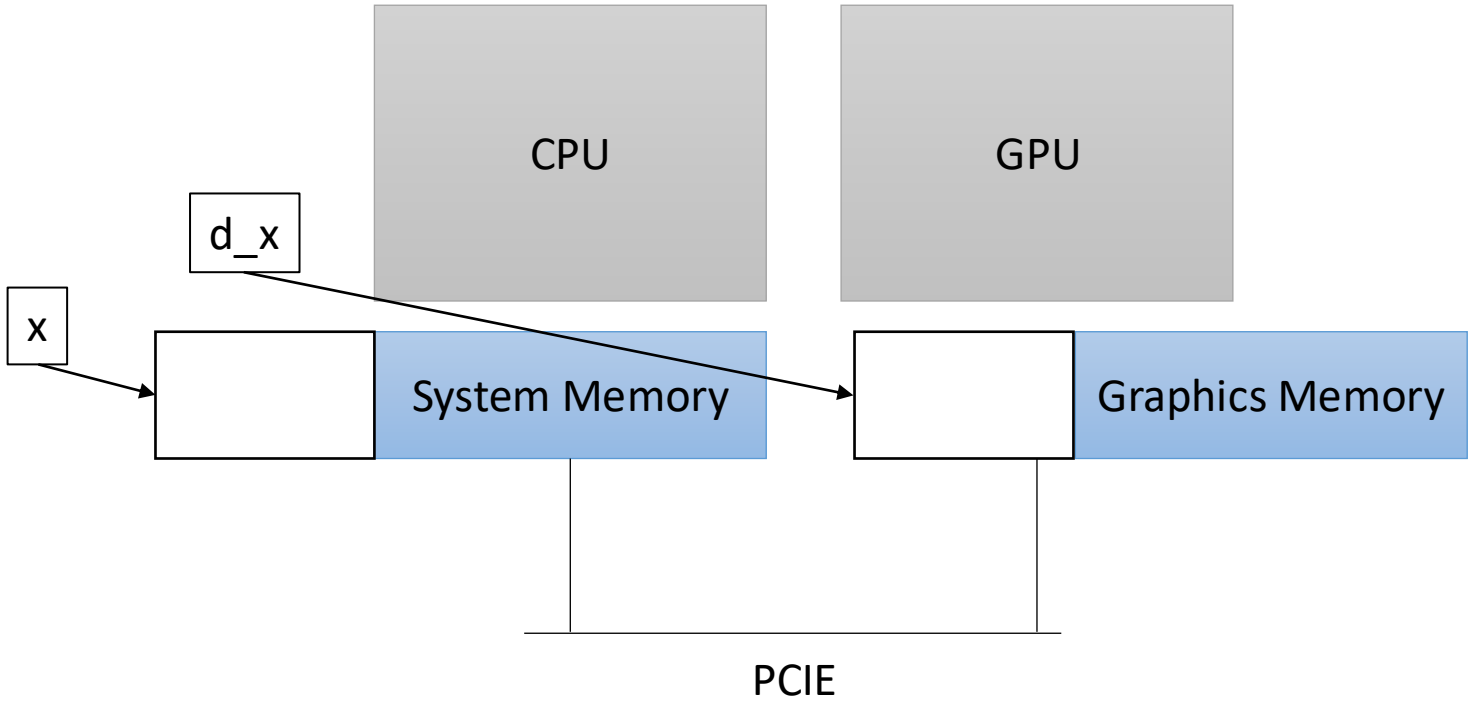
forms.gle/LjxX4hGcHhz4AfYg7

chart: docs.google.com/forms/d/e/1FAIpQLSfHfxGhnzTkZhUPARK6gMAjoYf8RmV8n-Xc4zJdaIioAr7ylQ/viewanalytics

GPU set up

- Our heterogeneous, parallel, programming model

Remember, GPU needs to access its own memory



The GPU Program

```
__global__ void vector_add(int * d_a, int * d_b, int * d_c, int size) {
    for (int i = 0; i < size; i++) {
      d_a[i] = d_b[i] + d_c[i];
   }
}
```

calling the function

```
vector_add<<<1,1>>>(d_a, d_b, d_c, size);
e |= cudaDeviceSynchronize();
```

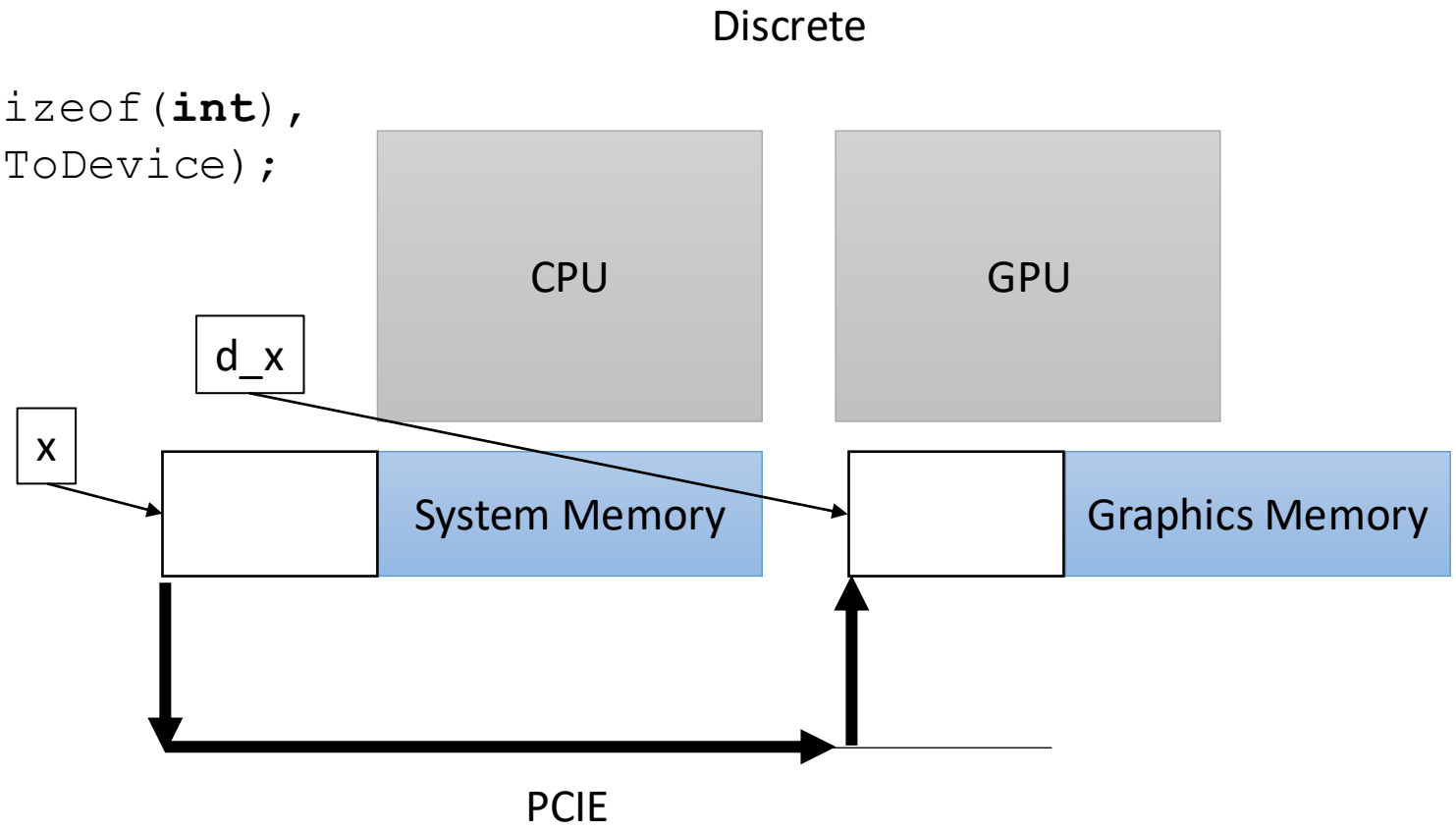
Constants can be passed in regularly

To make sure that it is finished

```
$ nvcc main.cu -o main
```

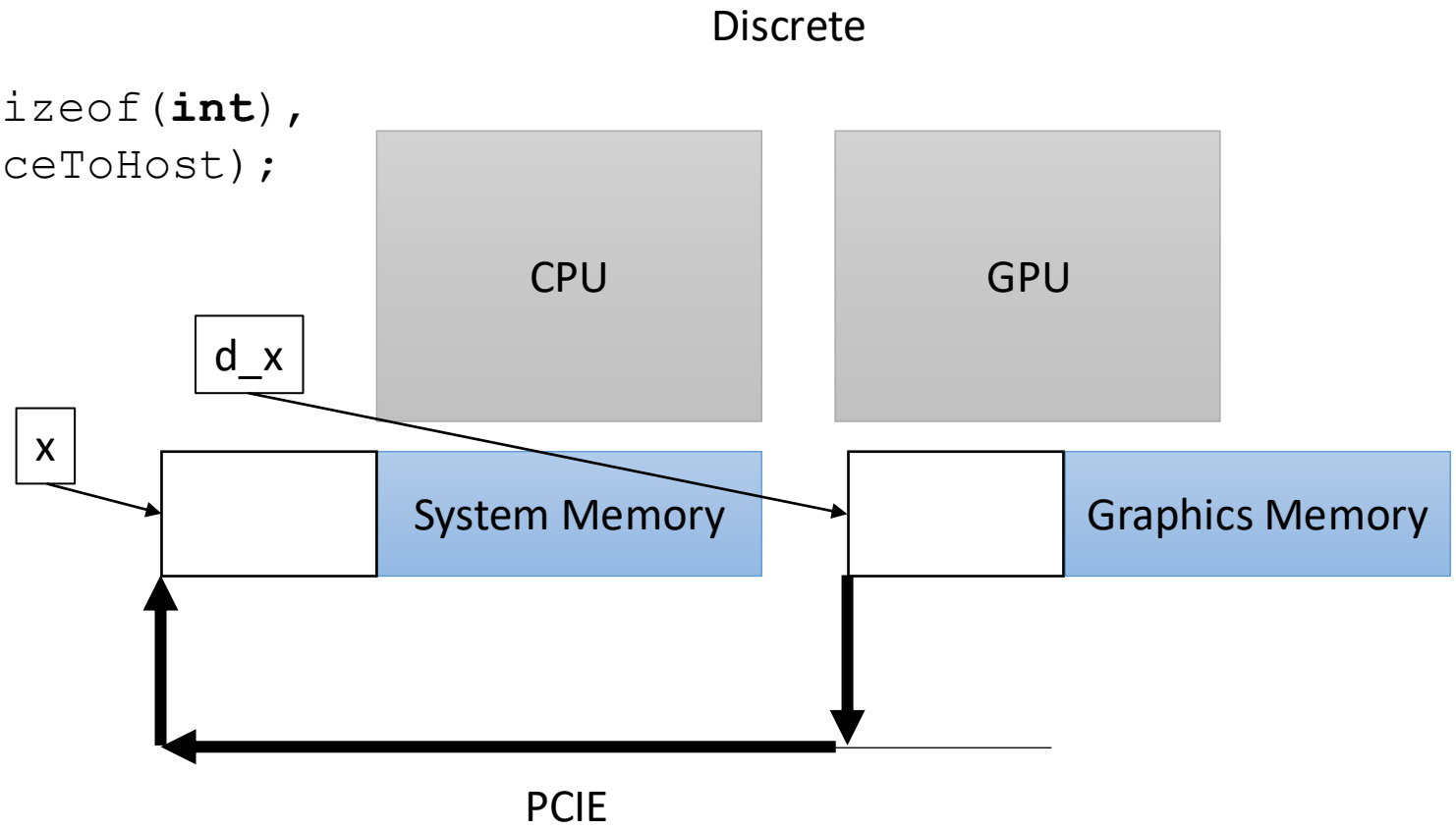
GPU set up

```
//initialize x on host
cudaMemcpy(d_x, x, SIZE*sizeof(int),
           cudaMemcpyHostToDevice);
```



GPU set up

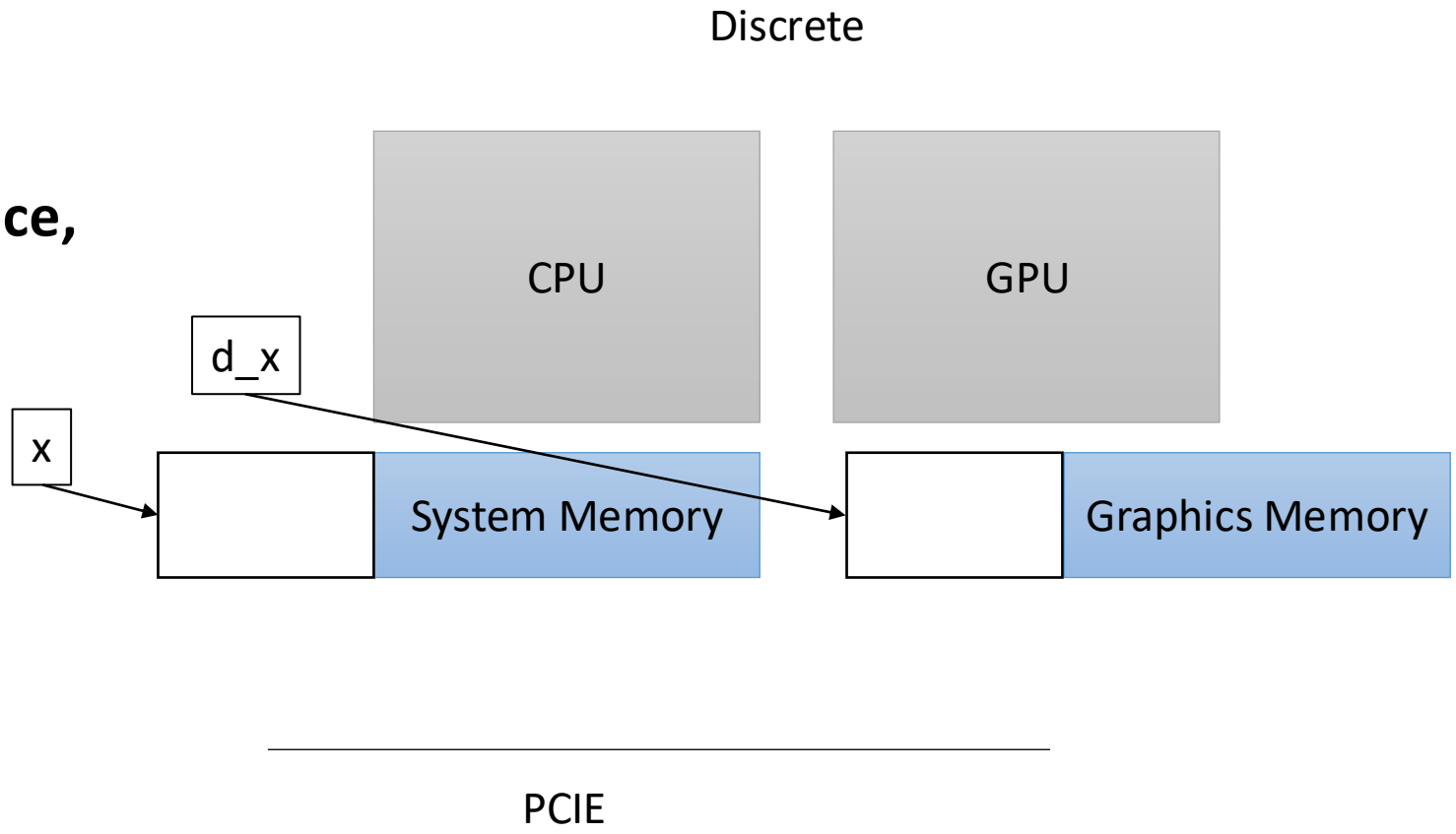
```
//initialize x on host
cudaMemcpy(x, d_x, SIZE*sizeof(int),
           cudaMemcpyDeviceToHost);
```



GPU set up

PCIe moves tens of GB/s while device memory moves TB/s,

Keep data on the device, copy rarely



Quiz 11.

How do we initialize memory for a variable we aim to use in the GPU computation?

Select one

A. Using cudaMemcpy

B. Using memcpy from C++

C. Just declaring a new variable



Scan to answer

forms.gle/RB5L3Q8BrcgwwaxB8

chart: docs.google.com/forms/d/e/1FAIpQLSc_FxDbMvqjPnhcKIB1RA0gGa6tezTQENINVbLqm5lAC62xiQ/viewanalytics